

Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky



Strojové učení pomocí knihoven Qiskit a sqlearn

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Studijní obor: Aplikovaná informatika

Autor: Andrei Nechiporuk

Vedoucí práce: Tomáš Kliegr

Praha, květen 2024

Prohlášení

Prohlašuji, že jsem bakalářskou práci "Strojové učení pomocí knihoven Qiskit a sqlearn" vypracoval samostatně za použití v práci uvedených pramenů a literatury

V Praze dne 06. květen 2024

.....

Podpis studenta

Poděkování

Chtěl bych vyjádřit své poděkování vedoucímu mé bakalářské práce, panu docentu Kliegrovi, za jeho spolupráci a neocenitelnou podporu během procesu psaní této práce.

Abstrakt

Bakalářská práce se zabývá využitím a porovnáním kvantových a klasických technik strojového učení, konkrétně se zaměřuje na klasifikační úlohy pro diagnostiku Parkinsonovy choroby s využitím současných frameworků, jako jsou qiskit a sqlearn pro kvantové strojové učení a scikit-learn pro klasické přístupy. Studie posuzuje, jak tyto technologie zvládají a analyzují datové soubory. Cílem je provést podrobný přehled a srovnání kvantových algoritmů oproti klasickým metodám se zaměřením na jejich výkonnost v klasifikačních úlohách, konkrétně s ohledem na evaluační metriky jako jsou přesnost, úplnost a F1-skóre. Práce je strukturována do různých etap, počínaje teoretickým úvodem do kvantového strojového učení, následovaným předzpracováním dat a výběrem příznaků pomocí vzájemné informace a implementací a hodnocením jak u klasifikátoru s podpůrnými vektory, tak jeho kvantového ekvivalentu, QSVC algoritmu. Tyto modely jsou testovány na dvou datových sadách a jejich výsledky jsou porovnávány s předchozími výzkumy. Práci uzavírá diskuse o srovnávacích výsledcích, kontrolní test s využitím datové sady "Oxford Parkinson's disease detection dataset" a zkoumání důležitých atributů a jejich vlivu na prediktivní výkonnost modelů.

Klíčová slova

kvantové strojové učení, klasické strojové učení, Parkinsonova nemoc, qiskit, sqlearn, scikit-learn

Abstract

The bachelor thesis deals with the use and comparison of quantum and classical machine learning techniques, specifically focusing on classification tasks for the diagnosis of Parkinson's disease. Using current frameworks such as qiskit and sqlearn for quantum machine learning and scikit-learn for classical approaches. The study assesses how these technologies handle and analyze datasets. The aim is to perform a detailed review and comparison of quantum algorithms versus classical methods, focusing on their performance in classification tasks, specifically with respect to evaluation metrics such as precision, recall and F1-score. The work is structured in different stages, starting with a theoretical introduction to quantum machine learning, followed by data preprocessing and feature selection using mutual information, and the implementation and evaluation of both the support vector machine classifier and its quantum equivalent, the QSVC algorithm. These models are tested on two datasets and the results are compared with previous research. The paper concludes with a discussion of the comparative results, a control test using the Oxford Parkinson's disease detection dataset, and an examination of the important attributes and their effect on the predictive performance of the models.

Keywords

quantum machine learning, classical machine learning, Parkinson's disease, qiskit, sqlearn, scikit-learn

Obsah

Úvod	12
1 Kvantové strojové učení	13
1.1 Přehled strojového učení a jeho význam v moderních aplikacích	13
1.2 Úvod do kvantových výpočtů a jejich potenciální dopad na počítačovou vědu	13
1.3 Definice kvantového strojového učení a jeho místo na průsečíku klasického ML a kvantové výpočetní techniky	14
1.4 Přehled existujících algoritmů kvantového strojového učení	15
1.4.1 QSVM algoritmus a jeho trénování na kvantových systémech	15
1.4.2 VQC algoritmus	15
1.4.3 QCNN algoritmus	16
1.5 Základy kvantových výpočtů	16
1.5.1 Qubity a jejich rozdíl od klasických bitů	16
1.5.2 Kvantové registry	16
1.5.3 Kvantové logické brány	17
1.5.4 Vysvětlení pojmu shots v prostoru kvantových výpočtů	18
1.5.5 Základní pochopení principu Quantum Feature Map v kontextu QML	18
2 Využití knihovny qiskit	20
2.1 Volba frameworku	20
2.2 Qiskit primitives	20
2.3 Qiskit Machine learning	20
2.4 Úvod do simulátorů od knihovny qiskit	21
3 Využití knihovny sqlearn	22
3.1 Úvod do sqlearn	22
3.2 Hlavní funkce sqlearn	22
3.3 Integrace s backendem	22
3.4 Kritické hodnocení	22
4 Dataset Early Biomarkers of Parkinson's Disease	23
4.1 Popis datasetu	23
4.2 Porozumění a příprava dat	23
4.3 Analýza dat	26
4.4 Použití kvantových algoritmů strojového učení	27
4.4.1 Rozdělení dat	27
4.4.2 Využití ZZFeatureMap	28
4.4.3 Vytvoření sampler metody s využitím simulátoru	28
4.4.4 Kvantové Jádro s Využitím Fidelity Statevector Kernelu	28
4.4.5 Využití QSVC algoritmu	29

4.4.6	Využití QSVC algoritmu od sqlearn	32
4.4.7	Využití klasického SVC algoritmu	33
4.5	Využití shap metody pro vysvětlení výstupů modelů strojového učení	35
4.5.1	QSVC algoritmus z knihovny qiskit	35
4.5.2	QSVC algoritmus z knihovny sqlearn	36
4.5.3	SVC algoritmus	36
4.6	Porovnávání algoritmů s předchozí studií	38
5	Dataset Oxford Parkinson's Disease Detection Dataset	39
5.1	Porozumění problematice datasetu	39
5.2	Vlastnosti dat	39
5.3	Popis jednotlivých atributů	39
5.4	Příprava dat	40
5.4.1	Načítání dat	40
5.4.2	Volba atributů	41
5.4.3	Korelační analýza	41
5.4.4	Vzájemná informace pro výběr příznaků	43
5.4.5	Analýza hlasových frekvencí pacientů	44
5.5	Kontrolní experiment na datasetu Oxford Parkinson's Disease Detection Dataset	46
5.5.1	Předzpracování a výběr prvků	46
5.5.2	QSVC model strojového učení knihovny qiskit	46
5.5.3	QSVC model strojového učení knihovny sqlearn	48
5.5.4	SVC model strojového učení knihovny scikit-learn	49
	Závěr	51
	Seznam použité literatury	52
	A Příprava dat pro Dataset Early Biomarkers of Parkinson's Disease	55
	B Vizualizace dat na datasetu Early Biomarkers of Parkinson's Disease	57
	C Rozdělení na trenovací a testovací sady a použití modelů strojového učení	58
	D Použití SHAP metody	64
	E Příprava dat pro Dataset Oxford Parkinson's Disease Detection Dataset	65
	F Rozdělení dat na trenovací a testovací sady a použití jednotlivých modelů	67
	G Použití QSVC algoritmu z knihovny sqlearn	71

Seznam obrázků

1.1 ZZFeatureMap

Zdroj: Vlatsní zpracování	19
4.1 Rozdělení cílového atributu 'status' v datové sadě Parkinson's Disease Dataset	
Zdroj: Vlatsní zpracování	24
4.2 Distribuce věku jednotlivých pacientů	
Zdroj: Vlatsní zpracování	26
4.3 poměr pohlaví účastníků	
Zdroj: Vlatsní zpracování	27
4.4 Matice záměn pro QSVC algoritmus	
Zdroj: Vlatsní zpracování	30
4.5 Matice záměn pro QSVC algoritmus, druhý experiment	
Zdroj: Vlatsní zpracování	31
4.6 Matice záměn pro QSVC algoritmus z knihovny squllearn	
Zdroj: Vlatsní zpracování	33
4.7 Matice záměn pro SVC algoritmus	
Zdroj: Vlatsní zpracování	34
4.8 Vizualizace force plot pro QSVC model z knihovny qiskit	
Zdroj: Vlatsní zpracování	35
4.9 Vizualizace force plot pro QSVC model z knihovny squllearn	
Zdroj: Vlatsní zpracování	36
4.10 Vizualizace force plot pro SVC model	
Zdroj: Vlatsní zpracování	37
5.1 Korelační matice	
Zdroj: Vlatsní zpracování	42
5.2 Vzájemná informace pro výběr příznaků	
Zdroj: Vlatsní zpracování	43
5.3 Rozdělení hlasových vlastností podle cílového atributu status	
Zdroj: Vlatsní zpracování	44
5.4 Rozdělení cílového atributu 'status' v datové sadě Oxford Parkinson's Disease Detection Dataset	
Zdroj: Vlatsní zpracování	45
5.5 evaluace qsvc modelu	
Zdroj: Vlatsní zpracování	46
5.6 analýza významnosti atributů qsvc modelu	
Zdroj: Vlatsní zpracování	47

5.7	evaluace qsvc modelu	
	Zdroj: Vlatsní zpracování	48
5.8	analýza významnosti atributů qsvc modelu	
	Zdroj: Vlatsní zpracování	49
5.9	evaluace svc modelu	
	Zdroj: Vlatsní zpracování	49
5.10	analýza významnosti atributů svc modelu	
	Zdroj: Vlatsní zpracování	50

Seznam tabulek

4.1	Popis atributů (Pinterová, 2023)	25
4.2	Výstup evaluace modelu QSVC z knihovny qiskit Zdroj: Vlatsní zpracování	32
4.3	Výstup evaluace modelu QSVC z knihovny sqlearn Zdroj: Vlatsní zpracování	33
4.4	Výstup evaluace modelu SVC z knihovny scikit-learn Zdroj: Vlatsní zpracování	34
4.5	Porovnávání naší studie s jinými studiemi na Parkinson's Disease datasetu	38
5.1	Ukázka Oxford Parkinson's Disease Detection datasetu Zdroj: Vlatsní zpracování	40

Seznam použitých zkratek

QML Quantum machine learning

QSVC Quantum support vector classifier

QNN Quantum neural network

SHAP Shapley additive explanations

SVC Support vector classifier

SVM Support vector machines

MNIST Modified national institute of
standards and technology

FMNIST Fashion - modified national

institute of standards and technology

QSVR Quantum Support Vector Regressor

VQC Variational Quantum Classifier

NISQ Noisy intermediate-scale quantum
era

PD Parkinson's disease

SMOTE Synthetic Minority Oversampling
Technique

ML Machine Learning

Úvod

Tato bakalářská práce se zabývá využitím a srovnáním kvantových a klasických přístupů v oblasti strojového učení, specificky pro klasifikační úlohy spojené s diagnostikou Parkinsonovy nemoci. V práci využíváme moderní frameworky, jako jsou qiskit a sqlearn pro kvantové strojové učení a scikit-learn pro klasické strojové učení. Výběr těchto technologií byl motivován jejich schopnostmi v oblasti využití a evaluace sofistikovaných modelů, které jsou schopné zpracovat a analyzovat komplexní datové sady.

Cílem této práce je provést systematický přehled a srovnání kvantových algoritmů s klasickými metodami, zaměřený na jejich výkon, přesnost a správnost při řešení klasifikačních úloh. Specificky se práce soustředí na implementaci a evaluaci SVC modelu a jeho kvantové varianty QSVC, přičemž oba přístupy jsou aplikovány na dvě datové sady "Parkinson's disease" a srovnávány s předchozími studiemi na stejné téma.

Metodologicky je práce rozdělena do několika fází. Po úvodním teoretickém přehledu kvantového strojového učení následuje fáze předzpracování dat na obou datasetech. Pak následuje fáze výběru relevantních rysů na základě vzájemné informace a implementace modelů QSVC a SVC. Výkon těchto modelů je poté evaluován pomocí metrik jako jsou F1-skóre, přesnost, úplnost, správnost a důležitost jednotlivých atributů je analyzována metodami jako SHAP a permutation importance.

1. Kvantové strojové učení

1.1 Přehled strojového učení a jeho význam v moderních aplikacích

V oblasti strojového učení je hlavním cílem vytvořit počítačové systémy, které budou schopny samostatně zlepšovat svoji funkcionalitu na základě získaných zkušeností. Ačkoli byly vyvinuty různé specializované učící se programy, širším cílem zůstává vytvoření systémů vybavených obecnějšími a silnějšími schopnostmi učení (T. Mitchell et al., 1990).

Nedávné studie upozornily na využití strojového učení ve zdravotnictví a ukázaly, že má potenciál zvýšit přesnost diagnostiky, optimalizovat léčebné plány a efektivněji předpovídat výsledky léčby pacientů (Esteva et al., 2019).

Studie "Machine learning applications in cancer prognosis and prediction" od Kourou et al. (2014), zdůrazňuje transformační dopad strojového učení na prognózu a predikci rakoviny. Vyzdvihuje schopnost strojového učení procházet komplexní, vysokorozměrné soubory dat a identifikovat kritické biomarkery a předpovídat trajektorie onemocnění. Techniky strojového učení, jako jsou metoda podpůrných vektorů, umělé neuronové sítě, bayesovské sítě a rozhodovací stromy prokázaly pozoruhodné úspěchy při modelování vývoje rakoviny a reakce na léčbu (Kourou et al., 2014).

1.2 Úvod do kvantových výpočtů a jejich potenciální dopad na počítačovou vědu

Kvantové výpočty, které se vyznačují změnou paradigmatu oproti klasickým výpočetním mechanismům, zahajují novou éru v počítačové vědě. Tento posun, který pochází z kvantové teorie informace, mění základní principy zpracování informací, umožňuje nakládat s informacemi v superpozicích a pomáhá provádět výpočty, které překračují klasická omezení (Sanders, 2023). Takové pokroky předznamenávají významný potenciál kvantové výpočetní techniky pro revoluci v datové vědě, která je podmnožinou počítačové vědy, díky zvýšení efektivity algoritmů a výpočetních schopností, zejména v oblasti kvantového strojového učení.

Sanders ve své studii poukazuje na prvotní impuls kvantové výpočetní techniky - simulaci kvantových systémů, což je úkol, na který klasická výpočetní technika nestačí. Z této rodící se myšlenky vzešly kvantové algoritmy, které se svou efektivitou nejen vyrovnávají, ale i předčí klasické výpočty a efektivněji řeší problémy, jako je faktorizace čísel a inverze funkcí. Vývoj kvantové výpočetní techniky v životaschopnou, i když teprve vznikající komerční technologii podtrhuje její potenciál jako převratné síly v technologickém prostředí (Sanders, 2023).

Ve výzkumné práci Pandey et al. (2015) se autoři podrobně zabývají úlohou kvantových počítačů při řešení rostoucích výzev analýzy velkých dat. Jádrem tohoto zkoumání je uznání

omezení, která přináší klasická výpočetní architektura při zvládnání exponenciálního nárůstu dat v různých odvětvích, včetně soc. médií, webových komunikačních nástrojů a dalších. Pandey a Ramesh vysvětlují transformační potenciál kvantových počítačů při zvyšování výpočetního výkonu, který je ztělesněn Moorovým zákonem (Pandey et al., 2015).

Ve studii zdůrazňují rodící se, ale eminentní dopad kvantových počítačů na analýzu velkých dat a poukazují na jejich schopnost způsobit revoluci v oblastech, jako je předpověď počasí, předpověď přírodních katastrof a business intelligence. Navzdory optimistickým vyhlídkám autoři uznávají technologické a environmentální překážky, kterým kvantové výpočty čelí, včetně potřeby teplot pro kvantové počítače blízkých absolutní nule a boje proti dekoherenci (Pandey et al., 2015).

1.3 Definice kvantového strojového učení a jeho místo na průsečíku klasického ML a kvantové výpočetní techniky

Kvantové strojové učení se stává slibným interdisciplinárním oborem, který spojuje výpočetní výkon kvantových technologií s metodikami strojového učení založenými na datech. Buffoni et al. (2021) tvrdí, že ML může významně těžit z pokroku kvantové výpočetní techniky a potenciálně přinést revoluci ve výpočetních úlohách prostřednictvím kvantových algoritmů. Očekává se, že tato synergie usnadní analýzu rozsáhlých souborů dat generovaných experimenty v oblasti kvantové fyziky, a tím zlepší prediktivní modelování a řízení experimentů (Buffoni et al., 2021).

Podstata strojového učení, která se vyznačuje schopností automatizovat učení a rozhodování na základě dat, nachází v kvantové oblasti nový rozměr. Integrace kvantových výpočtů je považována za prostředek k překonání výpočetních omezení, kterým čelí klasické modely strojového učení, zejména při zpracování velkého objemu dat (Buffoni et al., 2021). Kvantové algoritmy vyvinuté za tímto účelem mají za cíl urychlit trénování modelů strojového učení a slibují efektivitu přesahující možnosti tradičních výpočetních architektur.

Studie provedená Havenstein et al. (2018) porovnává výkonnost algoritmů strojového učení prováděných na klasických a kvantových výpočetních platformách. Podstata tohoto srovnání spočívá v posouzení, zda kvantové výpočty můžou zesílit účinnost nebo přesnost algoritmů strojového učení. Výzkum pečlivě zkoumá, jak si tyto kvantové algoritmy vedou ve srovnání se svými klasickými ekvivalenty s využitím konkrétních datových sad. Studie zejména zjišťuje případy, kdy kvantové algoritmy, zejména v oblasti problémů klasifikace více tříd, vykazují vyšší spolehlivost (Havenstein et al., 2018).

1.4 Přehled existujících algoritmů kvantového strojového učení

V této části ukážeme přehled kvantových algoritmů využitelných pro klasifikační úlohu strojového učení.

1.4.1 QSVM algoritmus a jeho trénování na kvantových systémech

QSVM algoritmus je rozšířením tradičního algoritmu SVM, který využívá principy kvantové výpočetní techniky ke zlepšení výpočetních procesů. QSVM využívá možností kvantového hardwaru i softwaru a jeho cílem je zlepšit účinnost klasických algoritmů SVM, které obvykle běží na GPU nebo CPU (Zeguendry et al., 2023).

QSVM začíná kódováním standardních dat, která často obsahují různé atributy, jako jsou třída, rysy a dimenze, poskytující základní údaje o kategoriích dat. Vzhledem k současným omezením kvantového hardwaru, zejména omezenému počtu qubitů, je nutné zjednodušit vlastnosti dat, aby byla zajištěna kompatibilita s dostupnými kvantovými výpočetními zdroji. Proto se běžně využívá analýza hlavních komponent, neboli PCA analýza, která redukuje rozměry dat (Zeguendry et al., 2023).

Po této redukci jsou klasická data transformována do formátu vhodného pro kvantové zpracování. Tato transformace zahrnuje tzv. "feature map", která slouží jako kvantová reprezentace atributů klasických dat. V dalším kroku se užívá algoritmus strojového učení pro optimalizaci výsledků klasifikace. Při tomto procesu se datová sada rozděluje na tréninkovou a testovací skupinu, určuje se počet potřebných qubitů a provádí se definování tříd dat. Navíc Zeguendry et al. (2023) zdůrazňují, že je třeba pečlivě stanovit parametry algoritmu, jako je počet iterací a hloubka obvodu, aby výsledek byl optimální (Zeguendry et al., 2023).

1.4.2 VQC algoritmus

Autoři Zeguendry et al. (2023) ve své práci uvádějí variační kvantový klasifikátor. Tento algoritmus používá ke klasifikaci binárních výsledků funkci $f(x, \theta) = y$, která je implementována na kvantovém počítači. Nejprve proběhne zakódování vstupních dat x do kvantového stavu pomocí obvodu, označeného jako S_x , přesněji zakódováním ve formě amplitudy stavu. Pak následuje aplikace dalšího kvantového obvodu U_θ , která tyto amplitudy zpracovává. V závěru probíhá jednoduché měření qubitů, které udává pravděpodobnost, že výstup bude buď "0" nebo "1" (Zeguendry et al., 2023).

Kromě toho Zeguendry et al. (2023) zdůrazňují, že parametry θ obvodu použitého v tomto klasifikačním procesu jsou trénovatelné, což umožňuje trénování pomocí tzv. "variační techniky" pro optimalizaci těchto parametrů.

1.4.3 QCNN algoritmus

Kvantové konvoluční neuronové sítě jsou kvantovými protějšky klasických konvolučních neuronových sítí používaných především v úlohách počítačového vidění. Je nutné upřesnit, že tyto kvantové modely jsou identifikovány spíše podle funkční podobnosti s klasickými CNN, jako je využití translační symetrie, než podle přísných matematických definic (Bowles et al., 2024).

Existuje několik kvantových modelů, které jsou funkčně paralelní s klasickými CNN, ale jsou přizpůsobeny rámci kvantového počítání. Jedním z takových modelů je QuanvolutionalNeuralNetwork, kde je konvoluční filtr pro vstupní vrstvu vyhodnocován náhodným kvantovým obvodem, který ke zpracování vstupních dat používá úhlové kódování. Další model, známý jako WeiNet, implementuje kvantovou konvoluční vrstvu pomocí metody 'amplitude embedding' a lineární kombinace jednotek. Cílem je snížit počet trénovatelných parametrů ve srovnání s tradičními CNN (Bowles et al., 2024).

1.5 Základy kvantových výpočtů

1.5.1 Qubity a jejich rozdíl od klasických bitů

Qubit, nebo kvantový bit, je základní jednotka informace v kvantové informatice. Na rozdíl od klasického bitu, který může nabývat pouze jedné ze dvou hodnot (0 nebo 1), qubit může existovat v jakémkoliv stavu kombinujícím tyto dvě hodnoty díky principu superpozice. Tento stav je možné matematicky vyjádřit jako lineární kombinaci stavů $|0\rangle$ a $|1\rangle$ pomocí komplexních koeficientů α a β :

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1.1)$$

kde $|\psi\rangle$ představuje kvantový stav qubitu, a α a β jsou amplitudy pravděpodobnosti, které splňují podmínku normalizace $|\alpha|^2 + |\beta|^2 = 1$. Tato podmínka zajišťuje, že celková pravděpodobnost nalezení qubitu v některém ze stavů při měření je vždy 1 (Michálek, 2023).

1.5.2 Kvantové registry

Kvantové počítače, podobně jako klasické počítače, neprovádějí výpočty pouze s jedním qubitem, ale skládajícím se z několika qubitů zvaným kvantový registr. Kvantový registr lze přirovnat k klasickému procesorovému registru, ve kterém kvantové výpočty probíhají skrze manipulaci s qubity v registru (Tomčala et al., 2021).

Každý qubit v takovém registru je v superpozici stavů $|0\rangle$ a $|1\rangle$, což umožňuje registr sestávající z n qubitů reprezentovat všechny možné kombinace 2^n bitů Tomčala et al. (2021).

Například registr ze dvou qubitů by měl 4 různé kombinace a následně by byl popsán jako:

$$|\psi_2\rangle = \alpha_0|00\rangle + \alpha_1|01\rangle + \alpha_2|10\rangle + \alpha_3|11\rangle$$

1.5.3 Kvantové logické brány

Klasické logické brány jsou odvozeny od Booleovské algebry, zatímco kvantové logické brány využívají podobných principů, avšak v kontextu kvantové mechaniky. Tyto brány jsou matematicky vyjádřeny pomocí unitárních maticových transformací, které se aplikují na kvantové registry tensorovým součinem s maticí reprezentující registr (Tomčala et al., 2021).

Hadamardův Operátor

Aplikace na $|0\rangle$ nebo $|1\rangle$ vytvoří superpozici s rovnoměrnou pravděpodobností.

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}}\langle 0| + \frac{|0\rangle - |1\rangle}{\sqrt{2}}\langle 1|$$

Pauliho Brány

Pauli X Brána

X (NOT brána)

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = |1\rangle\langle 0| + |0\rangle\langle 1|$$

Pauli Y Brána

Y (obracení a fázový posun)

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = i|1\rangle\langle 0| - i|0\rangle\langle 1|$$

Pauli Z Brána

Pauliho brána Z neguje amplitudu stavu $|1\rangle$ a ponechává amplitudu stavu $|0\rangle$ beze změny (Tomčala et al., 2021).

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = |0\rangle\langle 0| - |1\rangle\langle 1|$$

1.5.4 Vysvětlení pojmu shots v prostoru kvantových výpočtů

Termín **Shots** se v kontextu kvantového výpočtu odkazuje na počet měření kvantového stavu nebo obvodu, které jsou provedeny pro získání výsledků kvantového experimentu. V kvantovém strojovém učení optimalizace počtu střel může zlepšit efektivitu a rychlost tréninku tím, že se optimalizuje počet potřebných opakování pro dosažení přesných výsledků (Phalak et al., 2023).

V článku Phalak et al. (2023) od Koustubha Phalaka a Swaroopa Ghosha se zaměřují na optimalizaci počtu střel (shots) v kvantových strojových učících (QML) modelech s minimálním dopadem na výkonnost modelu. Pro demonstraci používají klasifikační úlohu s datovými sadami MNIST a FMNIST využívající hybridní kvantově-klasický QML model. Zkoumají, jak ovlivňuje počet střel trénink a testovací přesnost na celých a zkrácených verzích datových sad. Pozorují, že trénink na plné verzi datové sady poskytuje o 5-6% vyšší testovací přesnost než na zkrácené verzi s až 10x vyšším počtem střel pro trénink. To naznačuje možnost redukce velikosti datové sady pro urychlení tréninkového času.

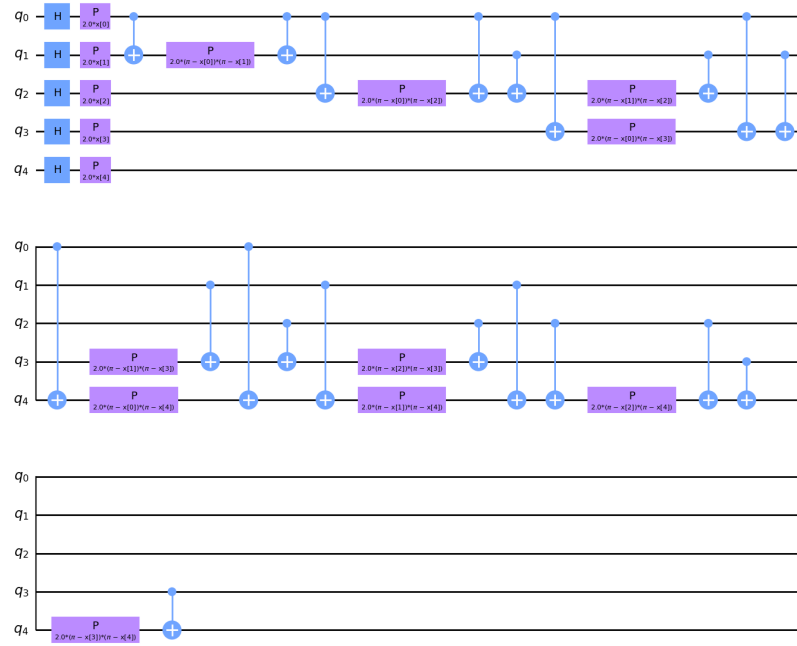
1.5.5 Základní pochopení principu Quantum Feature Map v kontextu QML

Kernelové metody jsou základní sadou algoritmů, které se obvykle používají při rozpoznávání vzorů a analýze dat. Tyto metody obvykle povyšují data do prostorů vyšších dimenzí, aby se zvýšila jejich struktura a separovatelnost. Tyto techniky mohou rozšířit reprezentaci dat do teoreticky nekonečných rozměrů pomocí tzv. kernelového 'triku' (Zeguendry et al., 2023).

V kvantové výpočetní technice je tento koncept rozšířen prostřednictvím metod kvantového kernelu, které provádějí nelineární transformace dat za účelem identifikaci hyperplochy, což je proces známý jako "feature map". Výzkumníci v této studii využili qiskit knihovnu k implementaci různých kvantových feature map, jako jsou ZZ, Z a Pauliho feature mapy, které obsahují Pauliho (X, Y, Z) hradla. Kromě toho autoři zkoumali vytváření vlastních feature maps, přizpůsobených specifickým požadavkům jejich datové sady (Zeguendry et al., 2023).

Konstrukce těchto kvantových feature maps často zahrnuje použití Hadamardových a entanglingových unitárních hradel. V jejich konkrétní implementaci byla použita ZZFeatureMap. Počet qubitů potřebných pro tyto procesy přímo souvisí s dimenzionalitou dat. Nastavením úhlu unitárních hradel se data zakódují do kvantového stavu (Zeguendry et al., 2023).

Níže je uveden obrázek ZZFeatureMap s pěti qubity



Obrázek 1.1: ZZFeatureMap

Zdroj: Vlastní zpracování

2. Využití knihovny qiskit

2.1 Volba frameworku

Pro účel implementace kvantových algoritmů byl zvolen framework Qiskit. Qiskit je open-source nástroj vyvíjený společností IBM, která je přední firmou v oblasti kvantového počítání. Tento nástroj je součástí projektu IBM Quantum, který disponuje velkou komunitou a postupem času se rozvíjí. V současné době do komunity IBM Quantum patří více než 210 členů společností z žebříčku Fortune 500, včetně laboratoří a univerzit (IBM, 2024a).

Táto platforma také nabízí školení a tutoriály pro pochopení jak kvantového počítání, tak i víceméně úzkého zaměření neboli kvantového strojového učení.

2.2 Qiskit primitives

Mitchell a Bello popisují qiskit primitives jako základní instrukce pro zpracování na dané úrovni abstrakce v rámci kvantových výpočtů. Tyto primitivy jsou stavebními kameny a fungují jako "černé skřínky", které provádějí potřebné operace, aniž by uživatel musel rozumět složitým detailům jejich implementace. Zavádějí dva základní typy primitiv jako Estimator a Sampler. Primitiv Estimator je určen k výpočtu hodnot očekávání pozorovatelných veličin na základě stavů připravených kvantovými obvody, zatímco primitiv Sampler počítá pravděpodobnosti bitových řetězců z kvantových obvodů. Primitiva nabízejí model, který kvantové programování přibližuje klasickému programování a více se zaměřuje na požadované výsledky (A. Mitchell et al., 2023).

2.3 Qiskit Machine learning

Qiskit Machine learning je vnořený balíček knihovny qiskit. Tento balíček zavádí základní výpočetní konstrukce, jako jsou kvantová jádra a kvantové neuronové sítě, které se používají v klasifikačních a regresních úlohách. Je navržen tak, aby byl uživatelsky přívětivý a umožňoval rychlé vytváření modelů, aniž by vyžadoval znalosti v oblasti kvantových výpočtů. Jednou z hlavních vlastností Qiskit Machine Learning je zavedení třídy 'FidelityQuantumKernel', která využívá algoritmus BaseStateFidelity. Tato funkce pomáhá provést přímý výpočet jaderných matic z datových sad, čímž zahajuje rychlejší řešení jak klasifikačních, tak i regresních problémů prostřednictvím integrace s Quantum Support Vector Classifier, neboli QSVC, anebo Quantum Support Vector Regressor (QSVR) (Qiskit, 2024).

Qiskit Machine Learning dále obohacuje své možnosti poskytováním algoritmů jako jsou NeuralNetworkClassifier a NeuralNetworkRegressor, které využívají kvantové neuronové sítě

v rámci klasifikace nebo regrese. Pro snadnější pochopení základů nabízí zjednodušené implementace, jako je VQC, neboli variační kvantová klasifikátor a variační kvantový regresor (Qiskit, 2024).

Balíček Qiskit Machine Learning je také integrovatelný s knihovnou PyTorch s pomocí TorchConnector (Qiskit, 2024).

2.4 Úvod do simulátorů od knihovny qiskit

BasicAer je součástí tzv. ekosystému qiskit a nabízí 3 simulátory. Tyto simulátory se liší v tom, jakým způsobem simulují kvantové obvody a jaké výsledky poskytují.

1. QasmSimulator: QASM je hlavním backendem pro qiskit aer. Tento nástroj simuluje provádění kvantových obvodů, jako by byly zpracovávány na skutečném kvantovém zařízení. Výstup pak je proveden ve formě počtů měření. Simulátor je vybaven univerzálními šumovými modely, včetně těch, které jsou automaticky aproximovány na základě kalibračních parametrů skutečného kvantového hardwaru (Wood, 2024).
2. StatevectorSimulator: Statevector simulátor je pomocný backend qiskit aer, který je navržen tak, aby ideálně simuloval kvantové obvody a po simulaci poskytoval konečný stavový vektor. Tento simulátor je přínosný zejména pro vzdělávací účely, teoretické zkoumání a ladění kvantových algoritmů (Wood, 2024). StateVector simulátor je naší volbou pro účel generování simulace, který bude podrobněji znázorněn v sekci číslo 4.4.
3. UnitarySimulator: Wood (2024) také upozorňuje na další simulátor, což je unitární simulátor. Tento backend se používá k simulaci finální unitární matice, kterou implementuje tzv. "ideální"kvantový obvod. Tento simulátor je také vhodným nástrojem, jako State vector simulátor pro vzdělávací účely a studium kvantových algoritmů (Wood, 2024).

3. Využití knihovny sqlearn

3.1 Úvod do sqlearn

Knihovna sqlearn, jak ji představili Kreplin et al. (2023), se manifestuje jako Python knihovna, která se používá pro účely kvantového strojového učení. Tato knihovna se bezproblémově integruje s klasickými nástroji a knihovny strojového učení, jako je scikit-learn. Tato knihovna využívá dvojvrstvou architekturu, která slouží jak výzkumníkům, tak praktikům v oblasti QML, a usnadňuje experimentaci a implementaci kvantových algoritmů (Kreplin et al., 2023).

3.2 Hlavní funkce sqlearn

Jednou z hlavních funkcí sqlearn je její obrovská sada nástrojů, která zahrnuje metody kvantových jader a kvantové neuronové sítě - tzv. "QNN", spolu se strategiemi kódování dat, automatizovanou správou spuštění a specializovanými technikami regularizace kvantových jader. Zaměření knihovny na kompatibilitu s NISQ ji staví jako most mezi současnými schopnostmi kvantového výpočtu a praktickými aplikacemi strojového učení (Kreplin et al., 2023).

3.3 Integrace s backendem

Integrace sqlearn s backendem usnadňuje spouštění kvantových algoritmů jak na simulovaných, tak na skutečných kvantových počítačích, přičemž využívá IBM Qiskit jako svůj rámec pro spouštění a správu obvodů. Tato vlastnost umožňuje spouštění algoritmů obsažených v sqlearn na hardware IBM a simulátorech založených na QIskitu (Kreplin et al., 2023).

3.4 Kritické hodnocení

Přestože sqlearn je jednou z poměrně užitečných knihoven v oblasti QML, má i své nedostatky. Jedním z hlavních omezení sqlearn je její závislost na ekosystému IBM Qiskit, což omezuje její použití s hardwarem jiných výrobců.

4. Dataset Early Biomarkers of Parkinson's Disease

4.1 Popis datasetu

Tento dataset obsahuje 130 pacientů, ze kterých 30 pacientů s Parkinsonovou chorobou (PD), 50 jedinců s poruchou chování ve fázi REM spánku (RBD), kteří jsou považováni za vysoce rizikové vývoje PD, a 50 zdravých kontrol. Účastníci se věnovali dvěma mluvním ukolům. Jedním z úkolů bylo dvakrát přečíst nahlas standardizovaný, foneticky vyvážený úryvek o 80 slovech. Druhým úkolem bylo přednést 90 sekundový monolog o osobních tématech, jako jsou jejich zájmy, rodina nebo práce (Hlavnička et al., 2017).

Rysy řeči byly extrahovány a analyzovány pomocí výpočetních metod podrobně popsanych Hlavnička et al. (2017).

4.2 Porozumění a příprava dat

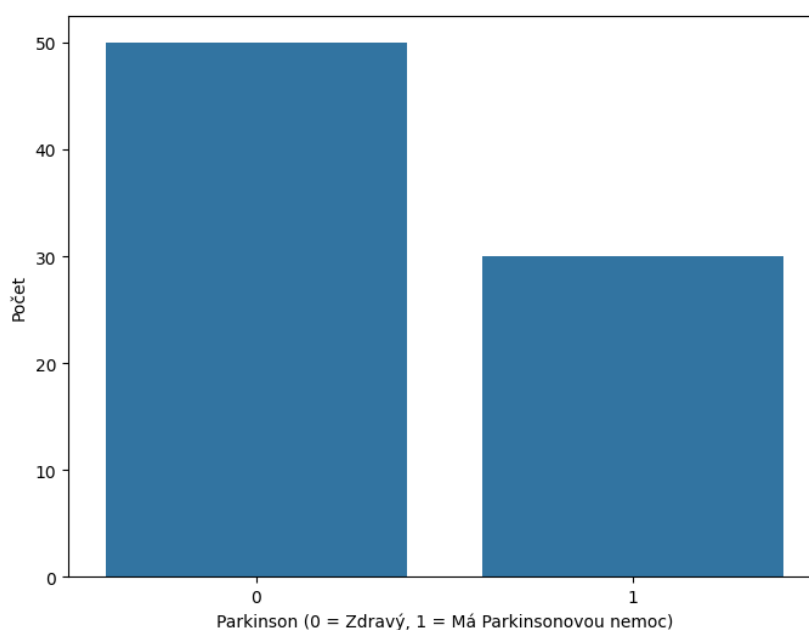
Soubor dat obsahuje chybějící hodnoty, které bylo potřeba vzít v úvahu při provedení dalších analytických procesů.

Při provedení této části jsme použili stejnou přípravu dat jako v práci Pinterová (2023), kde bylo nutné zredukovat množství atributů a odstranit nerelevantní kategorie, které se týkaly motorických funkcí respondentů.

Postup celé přípravy dat následoval stejnou logiku redukce a transformace dat s tím rozdílem, že místo R byl použit jazyk Python.

Důvodem pro použití stejné přípravy dat, jako bylo učiněno v práci Pinterová (2023), bylo to, že jsme se snažili využít shodný dataset pro srovnání výsledků modelů strojového učení. Tímto způsobem bylo možné zajistit komparabilitu výsledků algoritmů strojového učení.

Na níže uvedeném obrázku je zobrazen histogram rozdělení hodnot cílové proměnné Parkinson. Z histogramu je vidět, že 50 respondentů patří do kategorie 0, kde 0 naznačuje nepřítomnost Parkinsonovy nemoci. Do kategorie 1 spadají zbylých 30 respondentů, kde kategorie 1 ukazuje na přítomnost Parkinsonovy nemoci. Ve výsledku můžeme odvodit, že výsledná data nejsou vyvážená. Tato nevyváženost dat byla zachována pro shodu s daty použitými ve studii Pinterová (2023). V důsledku nevyváženosti dat jsme se rozhodli aplikovat metriky jako F1-skóre, úplnost a přesnost pro adekvátní hodnocení modelů.



Obrázek 4.1: Rozdělení cílového atributu 'status' v datové sadě Parkinson's Disease Dataset
Zdroj: Vlastní zpracování

Název atributu	Zkratka	Kategorie atributu		Datový typ	Popis
Participant code	Type	Unikátní atribut		Nominální	Unikátní kód určující zdravotní stav respondenta s unikátním číslem.
Age (years)	A	Demografické informace		Numerický	Věk respondenta v letech.
Gender	G	Demografické informace		Binární	Pohlaví respondenta.
Positive history of PD in family	PHP	Klinické	informace	Binární	Přítomnost PN v rodině.
Age of disease onset (years)	ADO	Klinické	informace	Numerický	Nástup nemoci v letech.
Duration of disease from first symptoms (years)	DDFS	Klinické	informace	Numerický	Délka nemoci od prvních symptomů.
Antidepressant therapy	AT	Medikace		Nominální	Užívání antidepresiv.
Antiparkinsonian medication	APM	Medikace		Binární	Užívání antiparkinsonik.
Antipsychotic medication	AP	Medikace		Binární	Užívání antipsychotik.
Benzodiazepine medication	BM	Medikace		Nominální	Užívání benzodiazepine.
Levodopa equivalent (mg/day)	LDE	Medikace		Numerický	Ekvivalent Levodopy.
Clonazepam (mg/day)	CL	Medikace		Numerický	Užívaná denní dávka Clonazepamu.
Entropy of speech timing	EST	Vyšetření	řeči: čtený projev	Numerický	Shannonova informační entropie vypočítaná z intervalů znělých, neznělých, pauzových a respiračních.
Rate of speech timing	RST	Vyšetření	řeči: čtený projev	Numerický	Rychlost intervalů znělých, neznělých a pauzových měřená jako sklon regresní přímky celkového počtu intervalů za daný časový úsek.
Acceleration of speech timing	AST	Vyšetření	řeči: čtený projev	Numerický	Průměrný rozdíl mezi RST úseku rozděleného na dvě poloviny s 25% překryvem.
Duration of pause intervals (ms)	DPI	Vyšetření	řeči: čtený projev	Numerický	Medián délky pauzových intervalů.
...	...	...		...	...

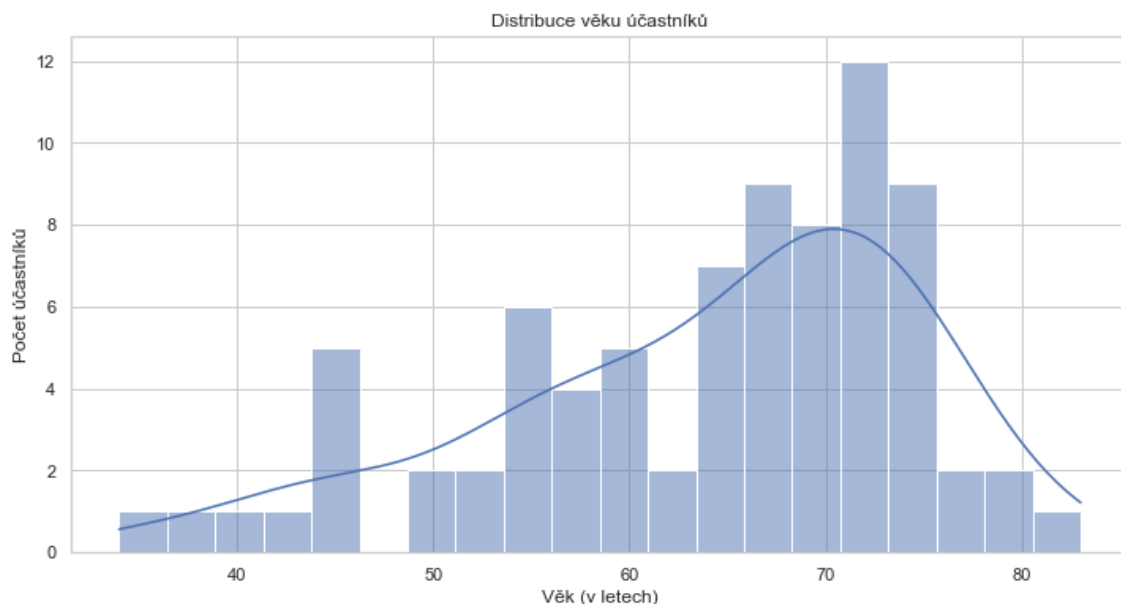
Tabulka 4.1: Popis atributů (Pinterová, 2023)

4.3 Analýza dat

1. Distribuce věku

S pomocí histogramu s křivkou hustoty pravděpodobnosti byla znázorněná distribuce věku jednotlivých účastníků.

Z histogramu je vidět, že věkový rozsah účastníků je široký, kolem 40 až po 80 let. Největší koncentrace účastníků se nachází v intervalu 60 až 75 let.

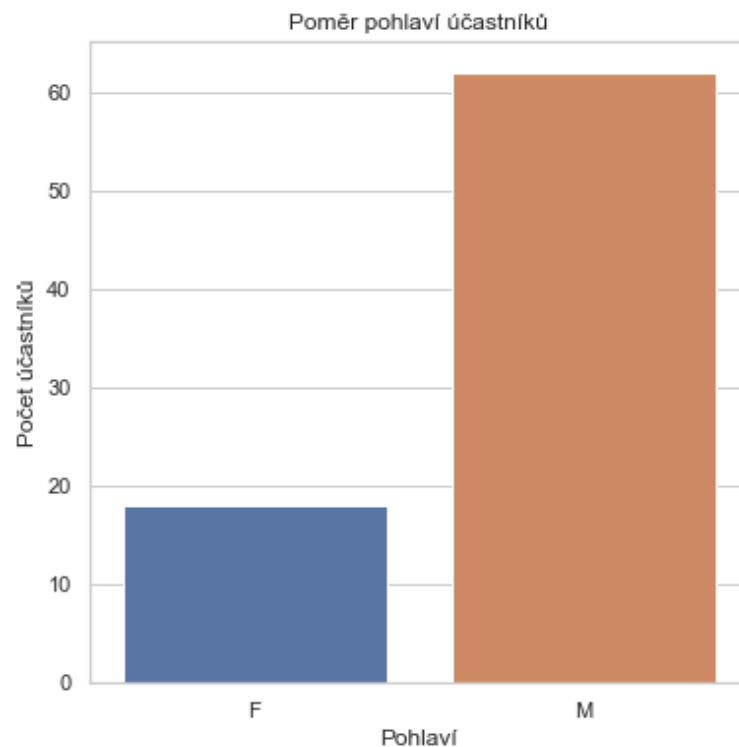


Obrázek 4.2: Distribuce věku jednotlivých pacientů

Zdroj: Vlastní zpracování

2. Poměr pohlaví

Sloupcový graf zobrazuje poměr pohlaví účastníků. Na základě tohoto grafu lze odvodit, že mužů zúčastněných ve výzkumu je podstatně více než žen.



Obrázek 4.3: poměr pohlaví účastníků

Zdroj: Vlatsní zpracování

4.4 Použití kvantových algoritmů strojového učení

V této kapitole se zaměříme na implementační postupy kvantových algoritmů strojového učení a ukážeme, jak mohou být tyto modely aplikovány na reálné datové sady. Poskytneme vysvětlení a demonstrace, jak lze kvantové algoritmy efektivně implementovat a využít pro zlepšení efektivity strojového učení.

4.4.1 Rozdělení dat

Nejprve s pomocí funkce `'train_test_split'` z knihovny `'scikit-learn'` jsme náhodně rozdělili data na trénovací a testovací sady. To pomáhá algoritmům učit se na jedné části dat, respektive trénovací sadě, a ověřovat výkonnost na části dat, která nebyla viděna během probíhajícího učení. S využitím parametru `'test_size=0.20'` jsme zajistili, že 80% dat bylo přiřazeno pro trénovací část a zbytek pro testování našeho modelu.

Další postup se opírá o použití tzv. metody 'MinMaxScaler', která normalizuje vstupní data do rozsahu (0,1). Normalizace zefektivňuje proces aplikaci kvantových obvodů a kvantových algoritmů strojového učení, neboť ty jsou značně citlivé na rozsah vstupních dat. Správně nastavené měřítko dat zabezpečuje to, že všechny vstupní atributy mají stejný význam pro algoritmus bez toho, aby byly nějaké atributy dominantní jen kvůli své škále.

Přístup rozdělení dat usnadňuje definování struktury kvantových obvodů. Počet qubitů v 'ZZFeatureMap' je závislý na počtu vlastností v trénovacích datech.

4.4.2 Využití ZZFeatureMap

V této části jsme se zaměřili na aplikaci třídy ZZFeatureMap z knihovny Qiskit, která transformuje klasická data do kvantového stavu.

Použitím ZZFeatureMap jsme zakódovali atributy do kvantového stavu. Parametr 'feature_dimension' odpovídá počtu atributů ve vstupních datech z našeho datasetu. Parametr 'reps' následně určuje, kolikrát se bude obvod opakovat.

V našem případě ZZFeatureMap byl nastaven s parametrem 'feature_dimension' na hodnotu 5 a parametrem 'reps' na hodnotu 3, což bylo zvoleno na základě optimalizace pro naši specifickou sadu dat a naše cíle výzkumu.

```
from qiskit.circuit.library import ZZFeatureMap
num_qubits = 5
feature_map = ZZFeatureMap(feature_dimension=num_qubits, reps=3)
```

4.4.3 Vytvoření sampler metody s využitím simulátoru

V této části našeho výzkumu jsme přistoupili k implementaci metody 'Sampler', která je integrována s vnořeným kvantovým simulátorem, založeným na knihovnách 'qiskit_ibm_runtime' a 'qiskit_aer' od open-source nástroje Qiskit.

Třída primitives, jejíž součástí je i Sampler metoda, je navržena tak, aby poskytovala jednoduché abstrakce pro běžné úlohy, jako je vzorkování kvantových stavů. Po návrhu této abstrakce se můžeme zaměřit na samotné algoritmy a aplikace bez nutnosti hlubokého ponoru do nízkourovňových detailů kvantového hardwaru (IBM, 2024b).

4.4.4 Kvantové Jádro s Využitím Fidelity Statevector Kernelu

V této práci bylo rozhodnuto použít Fidelity Statevector Kernel z toho důvodu, že tento koncept dovoluje rozšířit možnosti strojového učení prostřednictvím kvantových výpočtů, jak bylo podrobně popsáno ve studii Havlíček et al. (2019). Tento algoritmus byl navržen na

základě této studie. Práce Havlíček et al. (2019) zdůrazňuje, jak kvantové počítače mohou využít svůj velký stavový prostor pro řešení různých druhů problémů, které jsou pro klasické počítače výpočetně náročné, zejména pokud se jedná o kernelové metody.

Důvodem pro výběr jednoho z přístupů bylo využití výhod kvantového výpočetního prostoru pro zlepšení specifických výzev v doméně strojového učení, které jsme zkoumali.

4.4.5 Využití QSVC algoritmu

V této sekci je popsáno použití algoritmu QSVC s využitím kvantového jádra 'FidelityStatevectorKernel'. Pro transformaci vstupních dat byla nejprve aplikována metoda 'MinMaxScaler' pro normalizaci hodnot. S pomocí této metody jsme převedli hodnoty našich vstupních atributů do rozmezí od 0 do 1. Z normalizovaných dat bylo vybráno 12 určitých atributů, které jsou nejrelevantnější pro použití našeho algoritmu. Tyto atributy byly následně použity pro vytvoření trénovacích a testovacích sad s použitím funkce 'train_test_split'.

```
from sklearn.feature_selection import mutual_info_classif
import numpy as np
from sklearn.preprocessing import MinMaxScaler
X = df.drop(['Parkinson'],axis=1)
X_normalized = MinMaxScaler((0,1)).fit_transform(X)
X_normalized_df = pd.DataFrame(X_normalized, columns=X.columns)
columns_to_select = [ 'A', 'RST', 'AST', 'DPI', 'DVI', 'DUS', 'RST2', 'DPI2',
'DVI2', 'DUS2','DUF2', 'PIR2']
selected_features = X_normalized_df[columns_to_select]
y = df['Parkinson']

from qiskit_algorithms.utils import algorithm_globals
random_state = algorithm_globals.random_seed = 123
X_train, X_test, y_train, y_test = train_test_split(selected_features, y,
test_size=0.20, random_state=random_state)
```

Pro zakódování dat do kvantového stavu byl použit obvod 'ZZFeatureMap'. Tento obvod byl nakonfigurován takovým způsobem, aby odpovídal počtu qubitů rovnému počtu vybraných atributů, v našem případě dvanáct qubitů. Pro výpočet stavové věrnosti mezi kvantovými stavy bylo použito kvantové jádro 'FidelityStatevectorKernel'. Kvantové jádro bylo nastaveno s počtem tzv. 'shots' na 800.

Následně byl použit QSVC algoritmus s tímto kvantovým jádrem pro klasifikaci. QSVC algoritmus byl natrénován na trénovací sadě a výkonnost tohoto algoritmu byla ověřena pak na testovací sadě.

```
from qiskit.primitives import Sampler
from qiskit.circuit.library import ZZFeatureMap
```

```

from qiskit import QuantumCircuit
from qiskit_algorithms.state_fidelities import ComputeUncompute
from qiskit_machine_learning.kernels import FidelityStatevectorKernel
from qiskit.quantum_info import Statevector
from qiskit_machine_learning.algorithms import QSVC

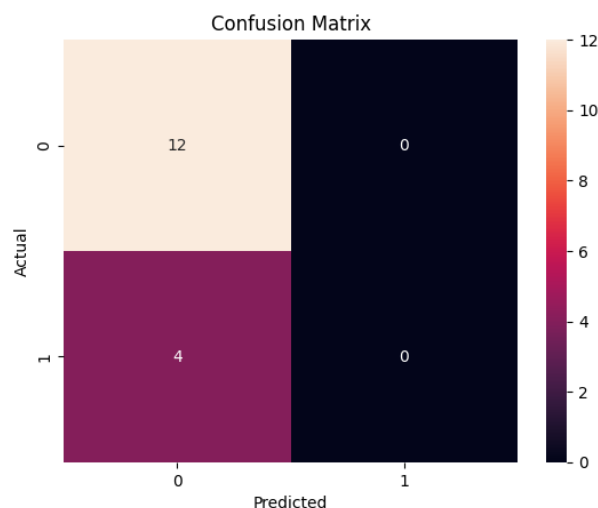
feature_map = ZZFeatureMap(feature_dimension=12, reps=3)

sampler = Sampler()
fidelity = ComputeUncompute(sampler=sampler)
parkinsons_kernel = FidelityStatevectorKernel(feature_map=feature_map,
statevector_type = Statevector, shots = 800,auto_clear_cache=True,)

qsvc_2 = QSVC(quantum_kernel=parkinsons_kernel,probability=True)
qsvc_2.fit(X_train, y_train)

```

Na základě výsledků klasifikace modelu QSVC prezentovaných na obrázku 4.4 vidíme, že model predikuje pouze třídu 0, tedy zdravé pacienty. Výpočty jednotlivých metrik nebylo nutné provádět, protože tento model nebyl schopen správně identifikovat třídu 1, respektive pacienty trpící Parkinsonovou chorobou. Tento problém poukazuje na nerovnováhu ve třídách, kde došlo k tzv. 'přeučení' na převažující třídě, což je 0.



Obrázek 4.4: Matice záměn pro QSVC algoritmus
Zdroj: Vlastní zpracování

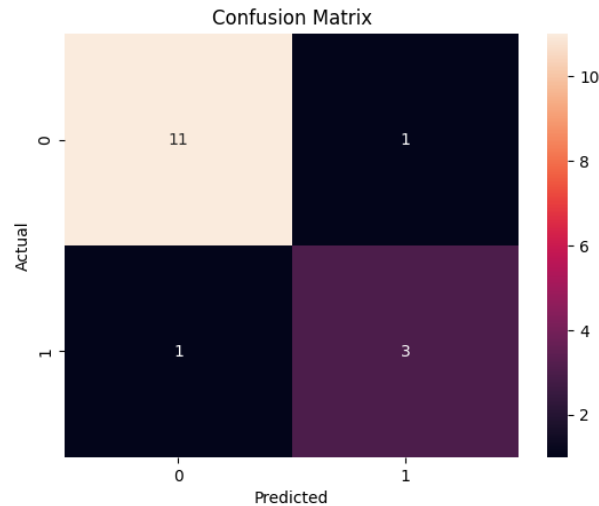
Proto bylo rozhodnuto provést další experiment na tomto algoritmu, kde bylo cílem prozkoumat chování daného algoritmu po redukcí počtu vstupních atributů na pět. Architektura nově vytvořeného algoritmu zůstává téměř nezměněná, tentokrát model byl trénován pouze na pěti qubitech namísto původních dvanácti.

Z níže uvedené tabulky můžeme konstatovat, že všechny metriky při výpočtu makro průměru dosahují hodnoty 0.833. Kromě toho, při výpočtu mikro průměru všechny metriky dosahují hodnoty 0.875. Pozorujeme také, že celková správnost je shodná s hodnotami metrik v mikro průměru a činí 0.875.

Stejné hodnoty metrik, jako jsou přesnost a úplnost se vyskytují z důvodu, že jmenovatele pro přesnost ($TP+FP$) i pro úplnost ($TP+FN$) jsou v našem případě identické, a proto mají stejné hodnoty. Skóre F-1, které je harmonickým průměrem přesnosti a úplnosti, bude stejné, pokud jsou obě tyto hodnoty identické, protože se jedná o průměr dvou stejných čísel.

```
feature_map = ZZFeatureMap(feature_dimension=5, reps=3)
sampler = Sampler()
fidelity = ComputeUncompute(sampler=sampler)
parkinsons_kernel = FidelityStatevectorKernel(feature_map=feature_map,
statevector_type = Statevector, shots = 800,auto_clear_cache=True,)

qsvc_2 = QSVC(quantum_kernel=parkinsons_kernel,
probability=True)
qsvc_2.fit(X_train, y_train)
```



Obrázek 4.5: Matice záměn pro Qsvc algoritmus, druhý experiment
Zdroj: Vlastní zpracování

Ukazatelé výkonnosti	Makro-průměr	Mikro-průměr	Hodnota
Přesnost	0.833	0.875	
Úplnost	0.833	0.875	
F1-Skóre	0.833	0.875	
Správnost			0.875

Tabulka 4.2: Výstup evaluace modelu QSVC z knihovny qiskit

Zdroj: Vlastní zpracování

4.4.6 Využití QSVC algoritmu od sqlearn

Tato část se podrobně zabývá použitím algoritmu QSVC od python knihovny sqlearn, která byla navržena ve Fraunhoferovém institutu pro výrobní techniku a automatizaci IPA Kreplin et al. (2023). Inicializace zahrnovala předzpracování dat, kde vstupní atributy jsou normalizovány pomocí 'MinMaxScaler', tento postup je stejný jako v předchozí sekci.

Následně jsme provedli zakódování dat do kvantového stavu s použitím kvantového obvodu 'YZ_CX_EncodingCircuit'. Tento obvod byl nastaven na 12 qubitů a 2 vrstvy.

Hlavní součástí tohoto použití je 'FidelityKernel', které bylo realizováno s využitím třídy Executor. Executor byl nakonfigurován pomocí 'BackendSampler' na simulátoru stavového vektoru 'Aer.get_backend("statevector_simulator")' od open-source nástroje Qiskit.

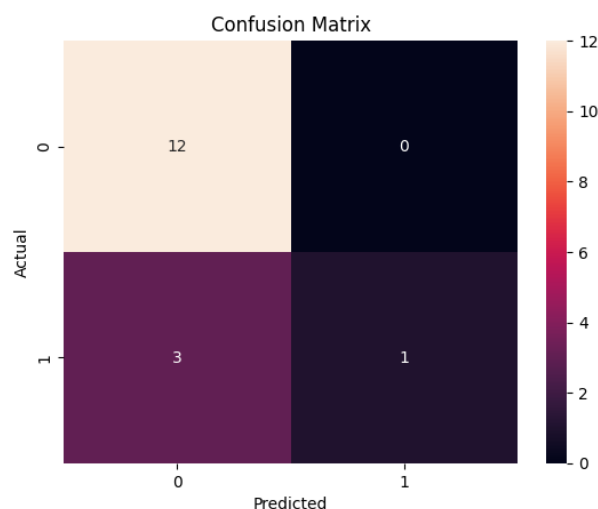
```
from qiskit.primitives import BackendSampler
from qiskit_aer import Aer
from sqlearn.encoding_circuit import (
    YZ_CX_EncodingCircuit)
from sqlearn.kernel import FidelityKernel, QSVC
from sqlearn.util import Executor

executor = Executor(BackendSampler(Aer.get_backend("statevector_simulator")))

feature_map = YZ_CX_EncodingCircuit(
    num_qubits=12, num_features=12, num_layers=2)
q_kernel = FidelityKernel(
    feature_map, executor=executor)

model_qsvc = QSVC(q_kernel, probability=True)
```

V závěrečné části nasazení algoritmu proběhlo trénování a evaluace modelu na základě trénovacích a testovacích sad.



Obrázek 4.6: Matice záměn pro QSV algoritmus z knihovny sqlearn
Zdroj: Vlatsní zpracování

Ukazatelé výkonnosti	Makro-průměr	Mikro-průměr	Hodnota
Přesnost	0.900	0.812	
Úplnost	0.625	0.812	
F1-Skóre	0.644	0.812	
Správnost			0.812

Tabulka 4.3: Výstup evaluace modelu QSV z knihovny sqlearn
Zdroj: Vlatsní zpracování

4.4.7 Využití klasického SVC algoritmu

Při použití klasického SVC algoritmu pro predikci Parkinsonovy nemoci bylo použito stejné předzpracování dat a výběr vstupních atributů jako v sekci 4.4.5.

Pro klasifikaci byl použit SVC algoritmus s přiřazeným argumentem 'probability=True'. Nastavení tohoto argumentu nám dovoluje po natrénování modelu vypočítat pravděpodobnostní skóre pro každou predikci, to je potřebné pro aplikaci metody SHAP, kterou se budeme zabývat v sekci 4.5.

Dále byl natrénován SVC na předzpracovaných trénovacích datech. Po natrénování modelu byly následně provedeny predikce na testovací sadě.

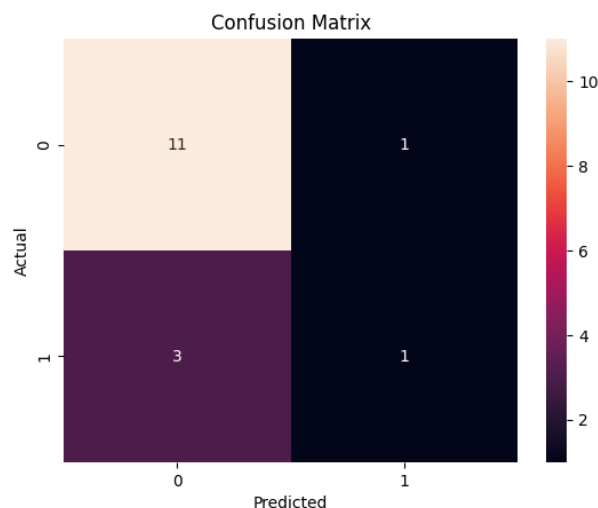
```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```

clf = make_pipeline(StandardScaler(), SVC(gamma='auto',probability=True))
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
score = clf.score(X_test, y_test)

```

Výsledné metriky a správnost modelu jsou vypočítány, aby se posoudila účinnost klasifikátoru v identifikaci případů Parkinsonovy nemoci. Výsledky klasifikace jsou prezentovány v tabulce 4.3 a obrázku 4.7.



Obrázek 4.7: Matice záměn pro SVC algoritmus

Zdroj: Vlastní zpracování

Ukazatelé výkonnosti	Makro-průměr	Mikro-průměr	Hodnota
Přesnost	0.643	0.750	
Úplnost	0.583	0.750	
F1-Skóre	0.590	0.750	
Správnost			0.750

Tabulka 4.4: Výstup evaluace modelu SVC z knihovny scikit-learn

Zdroj: Vlastní zpracování

4.5 Využití shap metody pro vysvětlení výstupů modelů strojového učení

V této kapitole jsme využili SHAP metodu. To je nástroj sloužící k interpretaci výstupů modelů, kde každému vstupnímu atributu přiřazuje hodnotu důležitosti pro určitou předpověď (Lundberg et al., 2017).

Pro vysvětlení predikce jsme použili konkrétní instanci, tedy první řádek testovací sady `X_test_df`, pomocí výrazu `'X_test_df.iloc[0, :]'`.

V této kapitole se zaměříme na tři modely jako jsou QSVC model z knihovny `qiskit`, klasický model SVC, a QSVC model z knihovny `squlearn`. U modelu QSVC z knihovny `qiskit` byla aplikována vizualizace SHAP force plot po natrénování na pěti attributech, což bylo z důvodu delší doby potřebné pro zpracování oproti ostatním modelům. Ostatní modely byly trénovány na dvanácti attributech.

4.5.1 QSVC algoritmus z knihovny qiskit

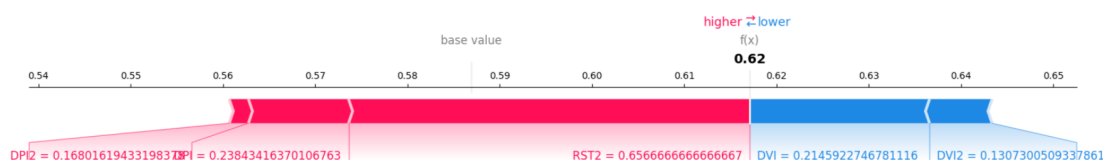
SHAP force plot

Na níže uvedeném obrázku byl znázorněn SHAP force plot, který vizualizuje naše atributy jako síly, které buď zvyšují, nebo snižují predikci modelu. Síla vlastností označená červenou šipkou zvyšuje pravděpodobnost predikce pozitivní třídy, modrá šipka naopak tuto pravděpodobnost snižuje.

Atribut RST2 má delší šipku než ostatní atributy, což reprezentuje velikost vlivu tohoto atributu. Po atributu RST2 následuje DPI a DPI2, které také ovlivnili model ke zvyšování skóre směrem doprava, tedy pozitivně přispívají k predikci pozitivní třídy.

Atributy DVI a DVI2 byly znázorněny modrou šipkou, tedy tyto atributy snižují pravděpodobnost predikce pozitivní třídy.

Po analýze jednotlivých atributů bych chtěl znázornit výstupní hodnotu $f(x)$, která je predikovanou hodnotou po zahrnutí všech příspěvků od všech atributů a činí 0.62.



Obrázek 4.8: Vizualizace force plot pro QSVC model z knihovny `qiskit`

Zdroj: Vlastní zpracování

4.5.2 QSVC algoritmus z knihovny squllearn

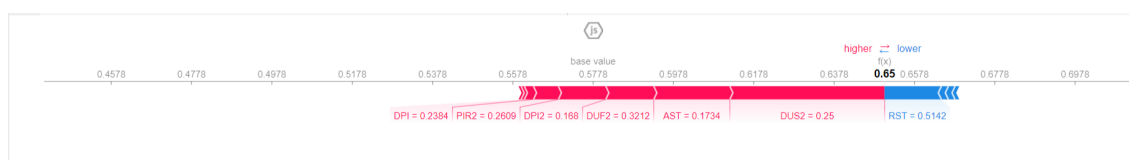
SHAP force plot

V této části jsme znázornili SHAP force plot, který zobrazuje atributy našeho modelu jako síly, které buď posilují, nebo oslabují predikci modelu. Síla atributů označená červenou šipkou zvyšuje pravděpodobnost predikce pozitivní třídy, na druhou stranu modrá šipka snižuje pravděpodobnost predikce pozitivní třídy.

Atributy DUS2 a AST mají nejdelší šipky, což reprezentuje velikost vlivu těchto atributů. Po attributech DUS2 a AST následují DUF2, DPI2, PIR2 a DPI, které také pozitivně přispěly k predikci pozitivní třídy.

Atribut RST byl znázorněn modrou šipkou. Tento atribut snižuje pravděpodobnost predikce pozitivní třídy.

Dále bych chtěl znázornit výstupní hodnotu $f(x)$, která představuje predikovanou hodnotu zahrnující příspěvky všech atributů a dosahuje hodnoty 0.65.



Obrázek 4.9: Vizualizace force plot pro QSVC model z knihovny squllearn

Zdroj: Vlatsní zpracování

4.5.3 SVC algoritmus

SHAP force plot

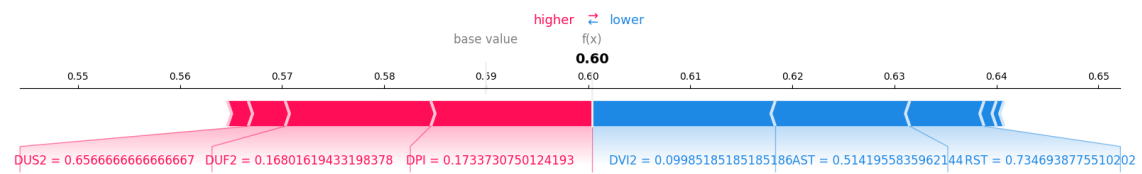
V této části byl také aplikován SHAP force plot pro ilustraci vlivu jednotlivých atributů na predikci modelu. Stejně jako v předchozích sekcích, na grafu červené šipky ukazují, že příslušný atribut zvyšuje pravděpodobnost pozitivní třídy, zatímco modré šipky naznačují snížení této pravděpodobnosti.

Atributy DPI a DUF2 mají delší šipky než ostatní atributy. V tomto případě jsou to nejvlivnější atributy a zvyšují pravděpodobnost predikce pozitivní třídy. Posledním atributem, který pozitivně přispívá k predikci pozitivní třídy, je DUS2.

Atributy jako jsou DVI2, AST a RST snižují pravděpodobnost predikce pozitivní třídy a jsou označeny modrými šipkami.

Na závěr této sekce bych rád uvedl hodnotu výstupu $f(x)$. Jak bylo zmíněno výše, výstupní hodnota $f(x)$ je predikovanou hodnotou po zahrnutí všech příspěvků od všech atributů a v

tomto případě dosahuje hodnoty 0.60.



Obrázek 4.10: Vizualizace force plot pro SVC model
Zdroj: Vlastní zpracování

4.6 Porovnávání algoritmů s předchozí studií

V rámci naší studie jsme se zaměřili na porovnání výkonnosti modelů strojového učení na datasetu Parkinsonovy choroby pomocí metriky F1 skóre. Modely, které byly v rámci naší studie vyhodnoceny, zahrnují klasický SVC a kvantové QSVC s použitím knihoven scikit-learn, qiskit a sqlearn. Také bylo provedeno srovnání s modelem Random Forest uvedeného ve studii Pinterová (2023).

Bylo rozhodnuto omezit trénování našich modelů pouze na 12 vybraných atributů u modelů QSVC od sqlearn a SVC algoritmu, a na 5 atributů u modelu QSVC z knihovny qiskit.

Z těchto modelů QSVC z knihovny qiskit dosáhl nejlepšího mikro F1 skóre 0.875 a makro F1 skóre 0.833 při použití 5 atributů. Výkonnost tohoto modelu byla vyšší než u Random Forest modelu, který dosáhl F1 skóre 0.807, ačkoli byl trénován na celém datasetu.

Model QSVC z knihovny sqlearn byl trénován na 12 attributech a dosáhl mikro F1 skóre 0.812 a makro F1 skóre 0.644. Navzdory použití více atributů bylo jeho skóre nižší než u QSVC modelu z knihovny qiskit.

Klasický SVC byl také trénován na 12 attributech a dosáhl však nižšího mikro F1 skóre 0.750 a makro F1 skóre 0.590. Výsledky signalizují, že tradiční algoritmus může být méně efektivní než kvantové modely i s větším počtem tréninkových atributů.

Celkově této výsledky přináší cenné informace o potenciálu modelů QSVC a SVC při analýze zdravotnických dat.

Následující tabulka zobrazuje mikro a makro F1-skóre dosažené jednotlivými modely.

Studie	Model strojového učení	F1 - skóre (mikro)	F1 - skóre (makro)	Počet atributů při trénování
Pinterová (2023)	Random Forest	0.807	N/A	celý dataset kromě cílového atributu
naš navržený model	QSVC	0.875	0.833	5
naš navržený model	QSVC (sqlearn)	0.812	0.644	12
naš navržený model	SVC	0.750	0.590	12

Tabulka 4.5: Porovnávání naší studie s jinými studiemi na Parkinson's Disease datasetu

5. Dataset Oxford Parkinson's Disease Detection Dataset

5.1 Porozumění problematice datasetu

Dataset o Parkinsonově chorobě, na který se odkazuje v publikaci "Suitability of Dysphonia Measurements for Telemonitoring of Parkinson's Disease" autorů Little et al. (2008), se skládá ze 195 hlasových záznamů od 31 osob, ze kterých 23 osob trpí Parkinsonovou chorobou (PD). Věk subjektů se pohyboval od 46 do 85 let, v průměru 65,8 let. Každý účastník poskytl v průměru šest fonací s dobou trvání od jedné do 36 sekund.

Nahrávání fonací proběhlo v kontrolovaném prostředí s použitím mikrofону, který byl umístěn na hlavě účastníků, což zajišťovalo vysokou kvalitu zvuku. Tyto nahrávky byly digitalizovány v 16 bitovém rozlišení a s frekvencí 44,1 kHz. Všechny vzorky byly rozděleny s pomocí digitální normalizaci amplitudy, aby bylo možné se soustředit na analýzu, která není ovlivněna změnami v hlasitosti řeči.

5.2 Vlastnosti dat

Dataset zahrnuje jak tradiční, tak netradiční metriky měření hlasu, jako jsou jitter, shimmer a poměr harmonických složek k šumu, stejně jako pokročilé metriky, jako je dimenze korelace (D2) a entropie hustoty periodických intervalů (RPDE).

5.3 Popis jednotlivých atributů

Tento dataset poskytuje rozsáhlou analýzu různých atributů hlasu, které mohou být využity při diagnostice Parkinsonovy nemoci. Každý účastník má svůj unikátní identifikační řetězec (name). Atributy jsou zaměřeny na různé aspekty hlasu a jeho frekvence, jako například:

- DVP:Fo(Hz), MDVP:Fhi(Hz), MDVP:Flo(Hz): Tyto parametry popisují střední, maximální a minimální základní frekvenci hlasu.
- MDVP:Jitter, MDVP:Jitter(Abs), MDVP:RAP, MDVP:PPQ, Jitter:DDP: Tyto ukazatele se týkají variability základní frekvence hlasu.
- MDVP:Shimmer, MDVP:Shimmer(dB), Shimmer:APQ3, Shimmer:APQ5, MDVP:APQ, Shimmer:DDA: Tyto metriky poskytují důležité informace o stabilitě a kvalitě hlasu.
- NHR, HNR: Jsou parametry používané k analýze hlasu, zejména při hodnocení kvality a zdraví hlasových schopností.

- status: Cílová proměnná indikující přítomnost (1), nebo absenci (0) Parkinsonovy nemoci.
- RPDE, DFA: Ukazatele spojené s opakováním v signálu a detrended fluctuation analysis.
- spread1, spread2, D2, PPE: Různé nelineární míry a míry složitosti signálu, včetně variability v periodách hlasu (Pitch Period Entropy).

Název	Popis	Datový typ
name	ASCII název subjektu a číslo záznamu	Numerický
MDVP:F0(Hz)	Průměrná základní frekvence hlasu	Numerický
MDVP:Fhi(Hz)	Maximální základní frekvence hlasu	Numerický
MDVP:Flo(Hz)	Minimální základní frekvence hlasu	Numerický
MDVP:Jitter(%), MDVP:Jitter(Abs), MDVP:RAP, MDVP:PPQ, Jitter:DDP	Několik měřítek variability v základní frekvenci	Numerický
MDVP:Shimmer, MDVP:Shimmer(dB), Shimmer:APQ3, Shimmer:APQ5, MDVP:APQ, Shimmer:DDA	Několik měřítek variability v amplitudě	Numerický
NHR, HNR	Dvě míry poměru šumu a tónu v hlasu	Numerický
status	Zdravotní stav subjektu (jeden) - Parkinsonova nemoc, (nula) - zdravý	Binární
RPDE, D2	Dvě nelineární dynamické složitosti míry	Numerický
DFA	Exponent fraktálního škálování signálu	Numerický
spread1, spread2, PPE	Tři nelineární míry fundamentální variability frekvence	Numerický

Tabulka 5.1: Ukázka Oxford Parkinson's Disease Detection datasetu

Zdroj: Vlastní zpracování

5.4 Příprava dat

5.4.1 Načítání dat

Následující kód importuje knihovnu pandas a načte soubor dat o Parkinsonově chorobě ze zadané adresy URL do rámce DataFrame:


```
import pandas as pd
url = "http://archive.ics.uci.edu/ml/machine-learning-
databases/parkinsons/parkinsons.data"
parkinsons_data = pd.read_csv(url)
```

5.4.2 Volba atributů

Do "X" jsou přiřazeny všechny vlastnosti kromě proměnných "name" a "status", jelikož proměnná "y" je cílovou proměnnou ("status"). Jak jsme zmiňovali v odstavci 6.3, "status" označuje přítomnost nebo nepřítomnost Parkinsonové nemoci.

```
X = parkinsons_data.drop(['name', 'status'], axis=1)
y = parkinsons_data['status']
```

5.4.3 Korelační analýza

Pro účel korelační analýzy byl nejprve přidán cílový atribut 'status' do proměnné 'X'. Pak proběhl výpočet korelační matice a následně byla vytvořena heatmapa pro zobrazení korelací mezi atributy. V posledním kroku atribut 'status' byl odstraněn z 'X' proměnné.

```
X['status'] = y

correlation_matrix = X.corr()

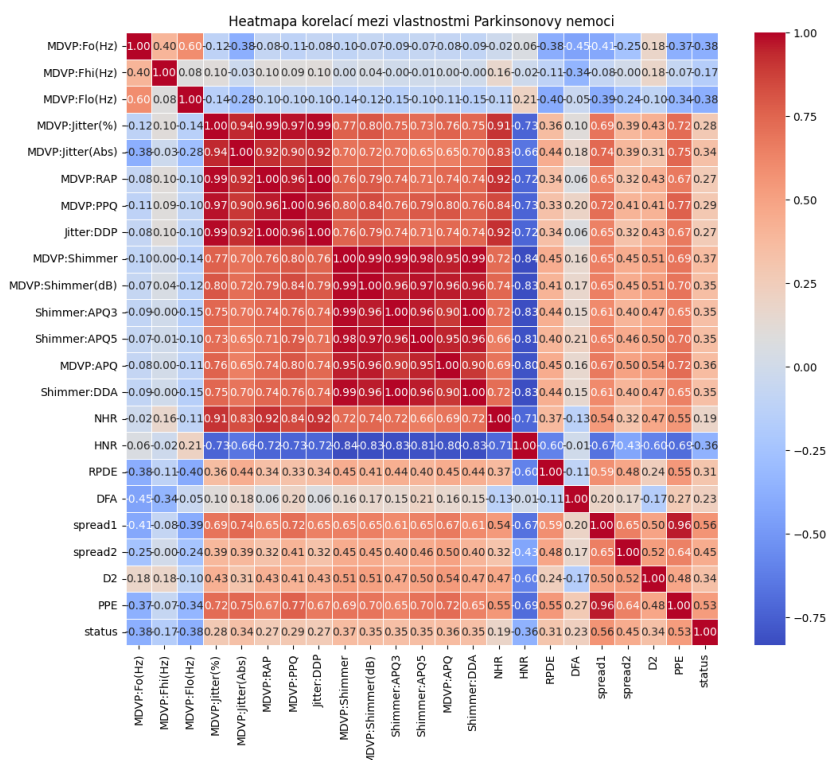
# Vytvoření heatmapy pro zobrazení korelací
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=.5)
plt.title('Heatmapa korelací mezi vlastnostmi Parkinsonovy nemoci')
plt.show()

X.drop('status', axis=1, inplace=True)
```

Graf 5.1 zobrazuje vztah mezi proměnnými, kde různé barvy odpovídají různým hodnotám korelace mezi těmito proměnnými. Barvy teplého spektra indikují silnou pozitivní korelaci, barvy studeného spektra ukazují na silnou negativní korelaci.

Pro zjištění, jak je každý atribut spojen s cílovým atributem status, se zaměříme na poslední sloupec. Z pozorování vyplývá, že atributy jako 'spread1', 'spread2', 'D2' a 'PPE'

mají silnou pozitivní korelaci s atributem 'status'. Na druhou stranu atributy jako 'MDVP:Fo(Hz)', 'MDVP:Fhi(Hz)', 'MDVP:Flo(Hz)' a 'HNR' mají negativní korelaci vůči atributu 'status'.



Obrázek 5.1: Korelační matice

Zdroj: Vlastní zpracování

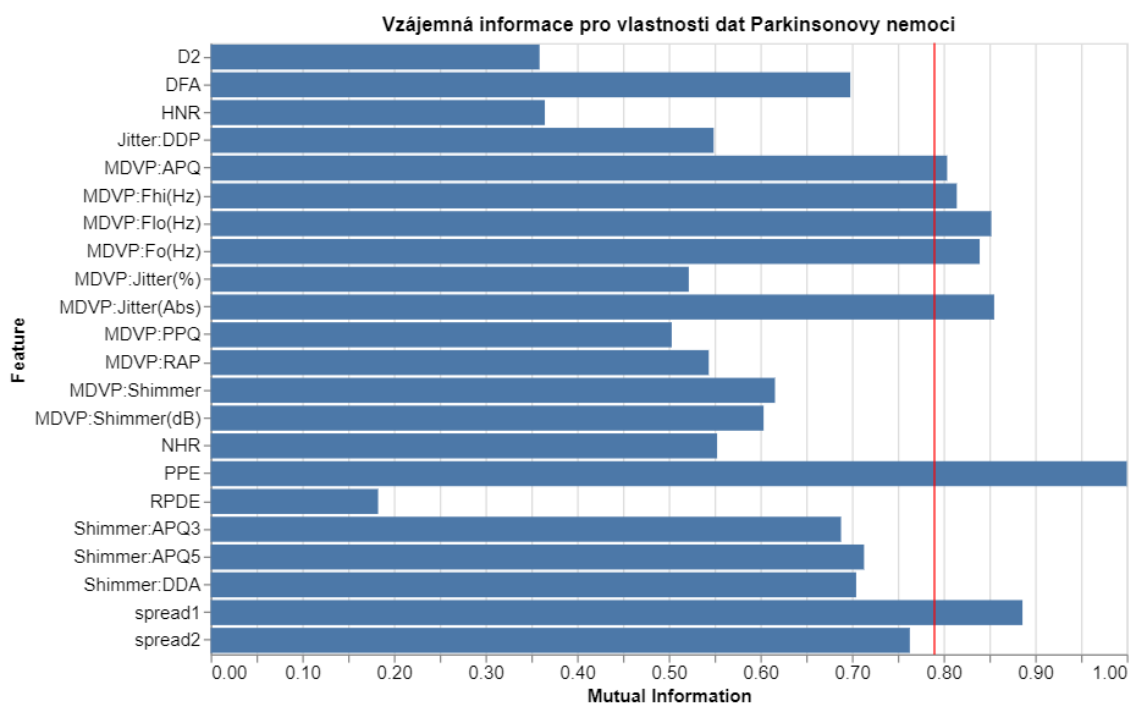
5.4.4 Vzájemná informace pro výběr příznaků

S pomocí této metody identifikujeme klíčové příznaky ze souboru hlasových měření, které jsou nejvíce relevantní pro predikci přítomnosti nebo absence této nemoci.

Pro každý atribut se vypočítá skóre vzájemné informace vůči cíli. Tato skóre jsou normalizována a jsou vybrány atributy, jejichž skóre je vyšší než hodnota (0.79).

```
from sklearn.feature_selection import mutual_info_classif
import numpy as np

mi = mutual_info_classif(X, y)
mi /= np.max(mi)
selected_features = X.columns[mi > 0.79]
```

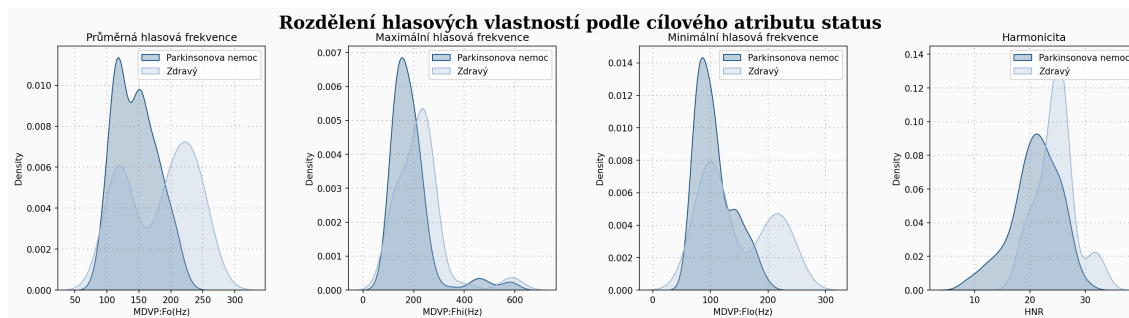


Obrázek 5.2: Vzájemná informace pro výběr příznaků

Zdroj: Vlastní zpracování

5.4.5 Analýza hlasových frekvencí pacientů

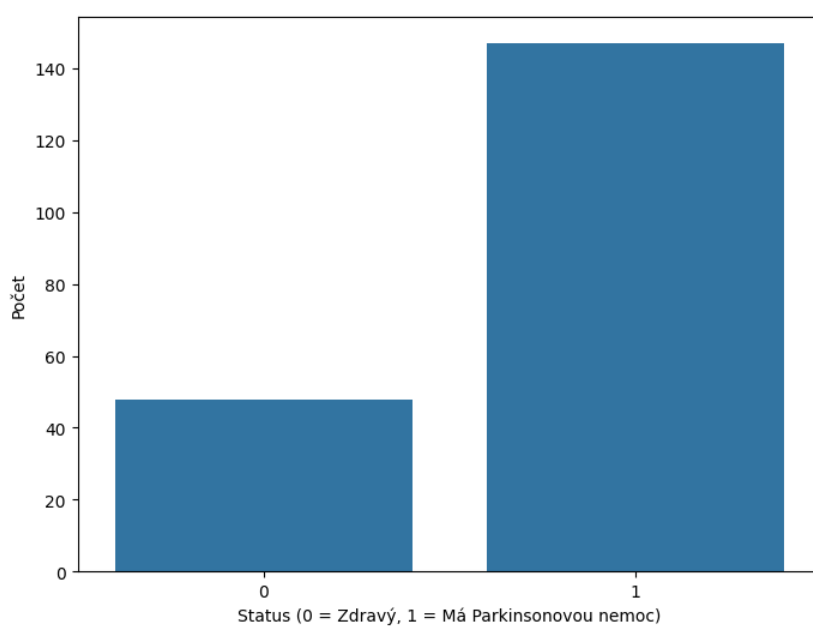
Níže uvedené grafy ilustrují rozdělení hlasových frekvencí mezi lidmi s Parkinsonovou nemocí a zdravými pacienty. Lze z těchto grafů odvodit, že průměrná hlasová frekvence pacientů s Parkinsonovou nemocí je více posunuta do nižších frekvenčních hodnot, než jsou hodnoty naměřené u zdravých osob.



Obrázek 5.3: Rozdělení hlasových vlastností podle cílového atributu status

Zdroj: Vlatsní zpracování

Rozdělení atributu status pro naši zvolenou datovou sadu ukazuje, že přibližně 75,38% je klasifikováno jako 1, což ukazuje na přítomnost Parkinsonovy nemoci. Přibližně 24,62% záznamů je klasifikováno jako 0, kde 0 naznačuje nepřítomnost Parkinsonovy nemoci. Zde vidíme, že dataset obsahuje výrazně více případů Parkinsonovy nemoci než případy zdravých pacientů. Na základě těchto počtů můžeme odvodit, že data nejsou vyvážená, jelikož je zde nepoměr mezi počtem osob s Parkinsonovou nemocí a zdravými osobami. Z hlediska strojového učení takováto nerovnováha může ovlivnit výkonnost klasifikačního binárního modelu, proto je potřeba buď použít metodu SMOTE, techniku pro vyrovnaní, anebo použít vážené metriky jako jsou F1-skóre, úplnost a přesnost.



Obrázek 5.4: Rozdělení cílového atributu 'status' v datové sadě Oxford Parkinson's Disease Detection Dataset

Zdroj: Vlastní zpracování

5.5 Kontrolní experiment na datasetu Oxford Parkinson's Disease Detection Dataset

5.5.1 Předzpracování a výběr prvků

Jak bylo zmíněno v sekci 5.4.5 tato datová sada trpí nevyvážeností tříd, to je běžný problém v lékařských datech, kde je jedna třída, v našem případě zdraví, méně zastoupena než třída nemoc. Abychom tento problém mohli vyřešit, museli jsme použít techniku syntetického převzorkování menšin, neboli SMOTE, abychom uměle rozšířili menšinovou třídu a zajistili, že náš model bude trénován na vyváženém souboru dat. Tímto přístupem jsme zmírnili zkreslení směrem k většinové třídě a zlepšili zobecnitelnost modelu.

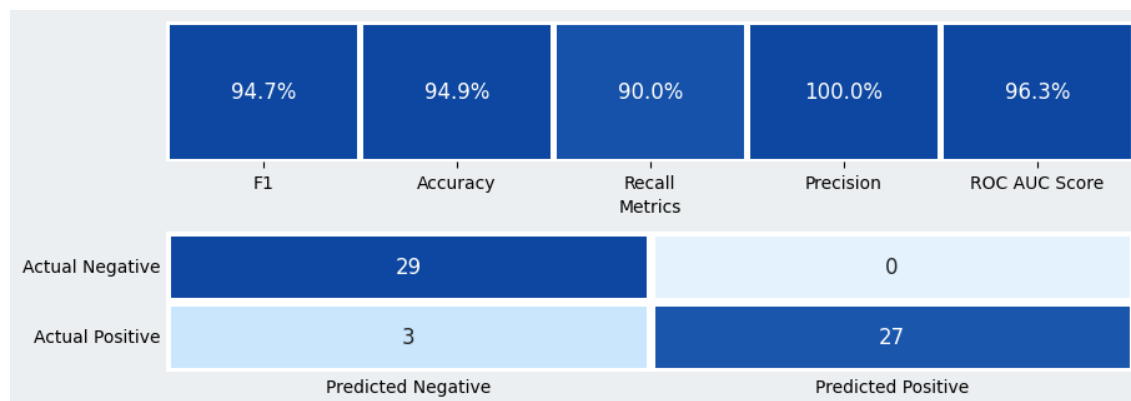
Po převzorkování byla pro každý rys vypočtena vzájemná informace a byly vybrány rysy s hodnotou vzájemné informace vyšší než 0,79. Popis tohoto přístupu je znázorněn v sekci 5.4.4.

5.5.2 QSVC model strojového učení knihovny qiskit

S vybranými prvky jsme přešli k nasazení přístupu kvantového strojového učení pomocí Qiskit. V kontrolním experimentu jsme stejně použili ZZFeatureMap pro transformaci klasických dat na kvantové stavy. Dále byl nasazen FidelityQuantumKernel pro měření podobnosti mezi těmito kvantovými stavy.

Po nasazení FidelityQuantumKernel byl nastaven pipeline, který zahrnuje MinMaxScaler a náš model QSVC. Po tomto kroku jsme natrénovali náš model a přešli k evaluační fázi, kde se testovala schopnost predikce modelu QSVC pomocí validační množiny.

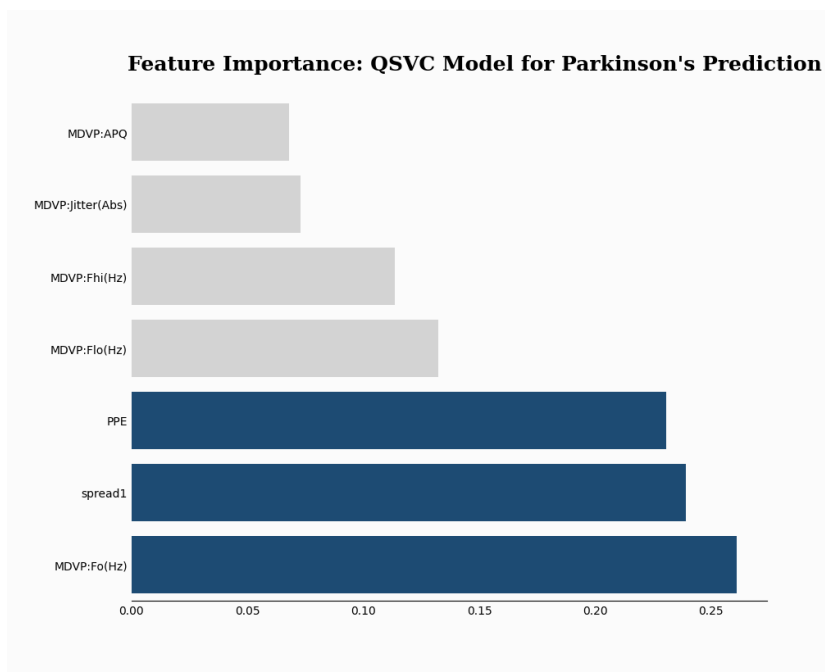
Pro posouzení výkonnosti modelu byly využity metriky jako jsou F1-skóre, úplnost, přesnost a další. Níže je uveden výstup evaluaci modelu.



Obrázek 5.5: evaluace qsvc modelu

Zdroj: Vlastní zpracování

V dalším kroku jsme provedli analýzu důležitosti atributů pro QSVC model za použití metody "permutation importance". Graf níže demonstruje hierarchické seřazení atributů vzhledem k jejich důležitosti pro predikci nemoci. Atributy jako jsou nelineární míry změny základní frekvence, což jsou 'PPE', 'spread1' a průměrná základní frekvence hlasu 'MDVP:Fo(Hz)', nejvíce ovlivnili QSVC model při predikci cílového atributu status.



Obrázek 5.6: analýza významnosti atributů qsvc modelu

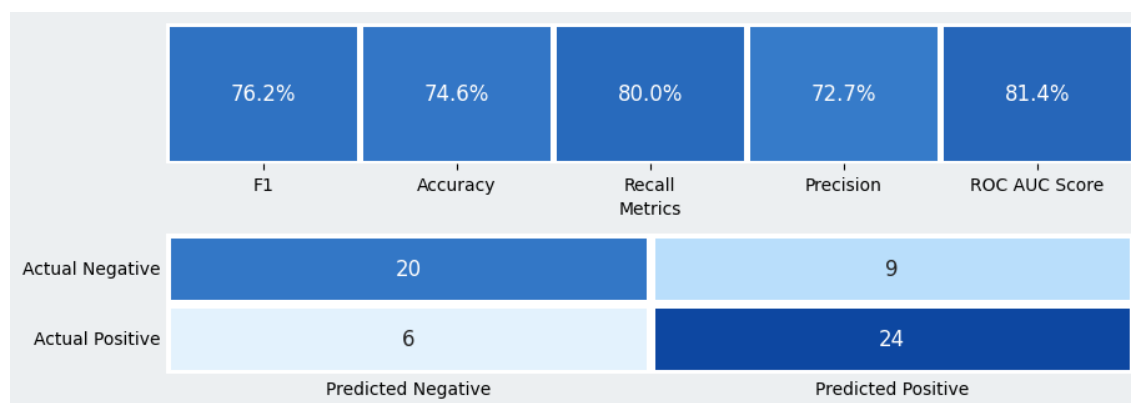
Zdroj: Vlastní zpracování

5.5.3 QSVC model strojového učení knihovny sqlearn

V prvním kroku nasazení modelu QSVC z knihovny sqlearn jsme načetli povinné moduly a provedli inicializaci prostředí s pomocí třídy Executor, v němž byl použit kvantový simulátor, respektive 'statevector_simulator'. Dále byla použita třída 'YZ_CX_EncodingCircuit' pro vytvoření kvantového obvodu se 7 qubity a 7 atributy a nastavením počtu vrstev na 2. Následujícím krokem bylo potřeba zajistit nasazení kvantového jádra, což bylo provedeno s pomocí 'FidelityKernel'.

Pak probíhá inicializace samotného QSVC modelu s pomocí předchozích uvedených kroků. Model byl natrénován a byla provedena predikce na validačních datech.

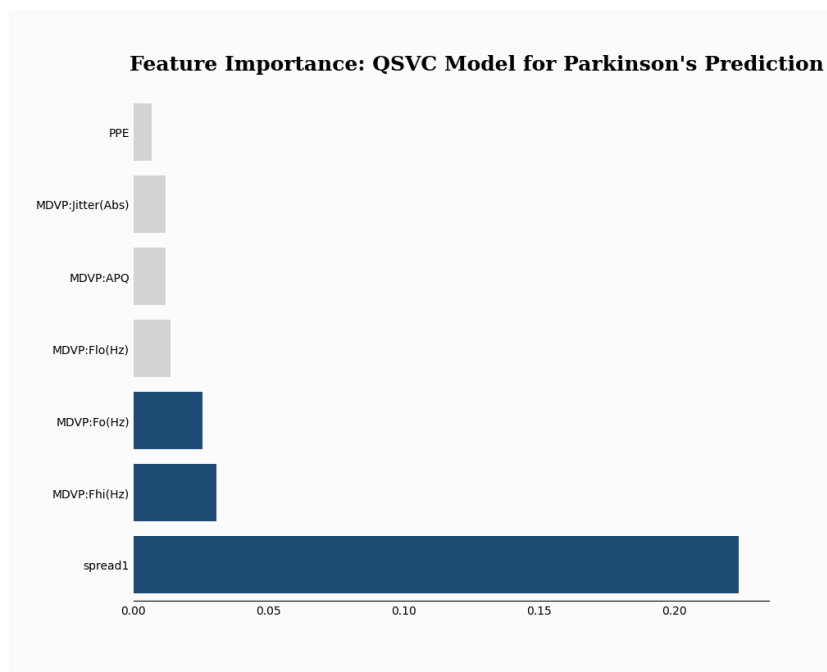
V závěrečné části nasazení modelu se získaná data z výstupu evaluaci modelu vynesou do grafu, kde budou zachyceny metriky, aby bylo možné kvantitativně posoudit úspěšnost predikcí.



Obrázek 5.7: evaluace qsvc modelu

Zdroj: Vlastní zpracování

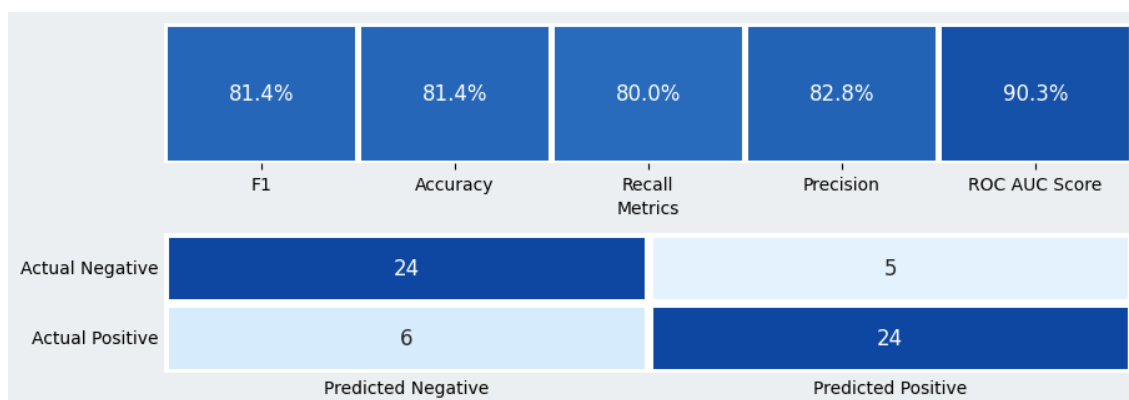
Bylo nezbytné provést v posledním kroku analýzu důležitosti atributů, což bylo provedeno obdobným způsobem jako v QSVC modelu z knihovny qiskit s pomocí metody "permutation importance". Na základě grafu 9.4 můžeme konstatovat, že atribut 'spread1' má největší vliv při predikci cílového atributu. Pak po něm následuje maximální základní frekvence hlasu, neboli atribut 'MDVP:Fhi(Hz)'. Atribut 'MDVP:Fo(Hz)', který byl identifikován jako třetí nejvlivnější ze všech níže uvedených atributů.



Obrázek 5.8: analýza významnosti atributů qsvc modelu
Zdroj: Vlastní zpracování

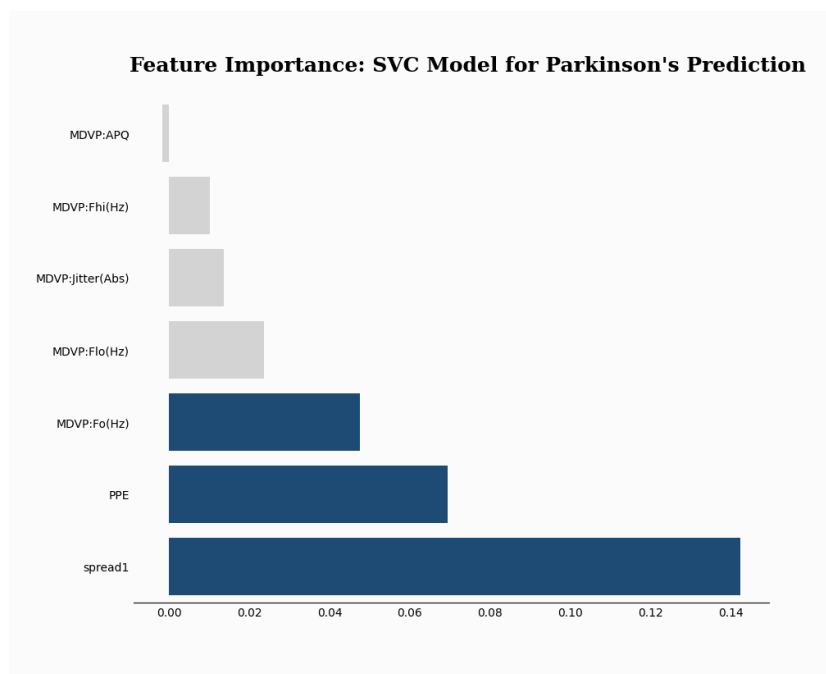
5.5.4 SVC model strojového učení knihovny scikit-learn

Podobně jako QSVC model z knihovny Qiskit byl klasický SVC integrován do pipeline, který zahrnuje MinMaxScaler. Pak proběhl proces trénování modelu SVC a jeho následná evaluace. Vyhodnocení modelu probíhalo stejným způsobem jako pro model QSVC a byly využity stejné metriky pro následné porovnání těchto modelů.



Obrázek 5.9: evaluace svc modelu
Zdroj: Vlastní zpracování

Dále jsme provedli analýzu důležitosti jednotlivých atributů s pomocí metody "permutation importance" z knihovny scikit-learn. Graf níže ilustruje seřazení atributů podle důležitosti. Jako nejvýznamnější se objevují 'spread1', 'PPE' a 'MDVP:F0(Hz)'. Atribut spread1 má nejvyšší skóre důležitosti, výrazně vyšší než ostatní atributy. Pak následuje PPE a MDVP:F0(Hz).



Obrázek 5.10: analýza významnosti atributů svc modelu

Zdroj: Vlastní zpracování

Závěr

V této bakalářské práci jsme se zabývali analýzou a porovnáním kvantových a klasických algoritmů strojového učení v kontextu diagnostiky Parkinsonovy choroby. Byly stanoveny tři hlavní cíle :

1. Vytvořit přehled kvantových algoritmů využitelných pro klasifikační úlohu strojového učení. Součástí bude popis jejich principu fungování, jak se liší v architektuře a trénování.
2. Implementace vybraného přístupu s využitím existujících frameworků.
3. Srovnání prediktivního výkonu vybraných knihoven na datasetu "Parkinson's disease".

Všechny cíle byly úspěšně splněny. Byl vytvořen a prezentován detailní přehled kvantových algoritmů využitelných pro klasifikační úlohu jako jsou QSVC, VQC a QCNN. Zejména u modelů QSVC byl detailně popsán jejich princip a rozdíly v architektuře a trénovacích procesech. Vybrané algoritmy, konkrétně QSVC a SVC, byly aplikovány s využitím frameworků qiskit, sqlearn a scikit-learn. Aplikace těchto algoritmů zahrnovala přípravu dat, výběr funkcí a evaluaci modelů na základě zvolených metrik. Srovnání kvantových a klasických přístupů ukázalo, že kvantové modely jsou schopné dosahovat srovnatelných nebo lepších výsledků v určitých aspektech prediktivního výkonu.

Navzdory úspěchům této práce je několik oblastí, které vyžadují další zkoumání. Jednou z nich je rozšíření využití kvantového strojového učení na další typy zdravotnických dat. Příkladem této oblasti je využití kvantových algoritmů strojového učení pro genetické sekvenování. Druhou oblastí je testování hybridních modelů, které integrují kvantové a klasické metody. Integraci lze rozšířit na experimentování, které testuje vliv různého počtu qubitů na výkonnost hybridních modelů.

Seznam použité literatury

- BOWLES, Joseph; AHMED, Shahnawaz; SCHULD, Maria, 2024. *Better than classical? The subtle art of benchmarking quantum machine learning models*. Dostupné z arXiv: 2403.07059 [quant-ph].
- BUFFONI, Lorenzo; CARUSO, Filippo, 2021. New trends in quantum machine learning(a). *Europhysics Letters*. Roč. 132, č. 6, s. 60004. Dostupné z DOI: 10.1209/0295-5075/132/60004.
- ESTEVA, Andre; ROBICQUET, Alexandre; RAMSUNDAR, Bharath; KULESHOV, Volodymyr; DEPRISTO, Mark; CHOU, Katherine; CUI, Claire; CORRADO, Greg; THRUN, Sebastian; DEAN, Jeff, 2019. A guide to deep learning in healthcare. *Nature Medicine*. Roč. 25. Dostupné z DOI: 10.1038/s41591-018-0316-z.
- HAVENSTEIN, Christopher; THOMAS, Damarcus; CHANDRASEKARAN, Swami, 2018. Comparisons of Performance between Quantum and Classical Machine Learning. *SMU Data Science Review*. Roč. 1, č. 4. Dostupné také z: <https://scholar.smu.edu/datasciencereview/vol1/iss4/11>.
- HAVLÍČEK, Vojtěch; CÓRCOLES, Antonio D.; TEMME, Kristan; HARROW, Aram W.; KANDALA, Abhinav; CHOW, Jerry M.; GAMBETTA, Jay M., 2019. Supervised learning with quantum-enhanced feature spaces. *Nature*. Roč. 567, č. 7747, s. 209–212. ISSN 1476-4687. Dostupné z DOI: 10.1038/s41586-019-0980-2.
- HLAVNÍČKA, Jan; CMEJLA, Roman; TYKALOVÁ, Tereza; SONKA, Karel; RŮŽIČKA, Evžen; RUSZ, Jan, 2017. Automated analysis of connected speech reveals early biomarkers of Parkinson's disease in patients with rapid eye movement sleep behaviour disorder. *Scientific Reports*. Roč. 7. Dostupné z DOI: 10.1038/s41598-017-00047-5.
- IBM, 2024a. *Join the IBM Quantum Network*. Dostupné také z: <https://www.ibm.com/quantum>. Datum přístupu: 2024-03-21.
- IBM, 2024b. *Qiskit*. Dostupné také z: <https://qiskit.org/documentation/>. Datum přístupu: 06.04.2024.
- KOUROU, Konstantina; EXARCHOS, Themis; EXARCHOS, Konstantinos; KARAMOUZIS, Michalis; FOTIADIS, Dimitrios, 2014. Machine learning applications in cancer prognosis and prediction. *Computational and Structural Biotechnology Journal*. Roč. 13. Dostupné z DOI: 10.1016/j.csbj.2014.11.005.
- KREPLIN, David A.; WILLMANN, Moritz; SCHNABEL, Jan; RAPP, Frederic; ROTH, Marco, 2023. *sQUlearn x2013 A Python Library for Quantum Machine Learning*. Dostupné z arXiv: 2311.08990 [quant-ph].
- LITTLE, MAX, 2008. *Parkinsons* [UCI Machine Learning Repository]. DOI: <https://doi.org/10.24432/C59C74>.

- LUNDBERG, Scott M; LEE, Su-In, 2017. A Unified Approach to Interpreting Model Predictions. In: GUYON, I.; LUXBURG, U. V.; BENGIO, S.; WALLACH, H.; FERGUS, R.; VISHWANATHAN, S.; GARNETT, R. (ed.). *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., s. 4765–4774. Dostupné také z: <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>.
- MICHÁLEK, Jan, 2023. Aplikace geometrických algeber v kvantovém počítání. *Kvaternion*. Roč. 1, č. 1-2, s. 85–111. Dostupné také z: http://kvaternion.fme.vutbr.cz/2023/kv23_1-2_michalek_web.pdf.
- MITCHELL, Abby; BELLO, Luciano, 2023. *What are Qiskit Primitives?* [<https://medium.com/qiskit/what-are-qiskit-primitives-9bf63c1eacc7>]. Accessed: 2024-04-10.
- MITCHELL, T; BUCHANAN, B; DEJONG, G; DIETTERICH, T; ROSENBLOOM, P; WAIBEL, A, 1990. Machine Learning. *Annual Review of Computer Science*. Roč. 4, č. Volume 4, 1990, s. 417–433. ISSN 8756-7016. Dostupné z DOI: <https://doi.org/10.1146/annurev.cs.04.060190.002221>.
- PANDEY, Abhishek; RAMESH, V, 2015. Quantum computing for big data analysis. *Indian Journal of Science*. Roč. 14, č. 43, s. 98–104.
- PHALAK, Koustubh; GHOSH, Swaroop, 2023. *Shot Optimization in Quantum Machine Learning Architectures to Accelerate Training*. Dostupné z arXiv: 2304.12950 [quant-ph].
- PINTEROVÁ, Lucie, 2023. *Využití strojového učení pro predikci Parkinsonovy nemoci [online]*. Dostupné také z: <https://theses.cz/id/ssa221/>. SUPERVISOR: Tomáš Kliegr.
- QISKIT, 2024. *Qiskit Machine Learning Documentation*. Dostupné také z: <https://qiskit-community.github.io/qiskit-machine-learning/>. Accessed: 2024-03-21.
- SANDERS, Barry C., 2023. Quantum computing for data science. *Journal of Physics: Conference Series*. Roč. 2438, č. 1, s. 012007. ISSN 1742-6596. Dostupné z DOI: 10.1088/1742-6596/2438/1/012007.
- TOMČALA, Jiří; LAMPART, Marek, 2021. Prvodce implementace v IBM Qiskit s úvodem do kvantového počítání.
- WOOD, Christopher J., 2024. *Introducing Qiskit Aer: A high performance simulator framework for quantum circuits*. Medium. Dostupné také z: <https://medium.com/qiskit/qiskit-aer-d09d0fac7759>. Accessed on: 2024-03-28.
- ZEGUENDRY, Amine; JARIR, Zahi; QUAFAFOU, Mohamed, 2023. Quantum Machine Learning: A Review and Case Studies. *Entropy*. Roč. 25, č. 2. ISSN 1099-4300. Dostupné z DOI: 10.3390/e25020287.

Přílohy

A. Příprava dat pro Dataset Early Biomarkers of Parkinson's Disease

```
1 import pandas as pd
2
3 #V těchto krocích provádíme stejnou přípravu dat jako v práci Pinterova(2023)
4   , abychom mohli porovnat výkonnost modelu strojového učení
5
6 import pandas as pd
7 #Načtení datasetu
8 dataset = pd.read_csv(r'dataset (1).csv')
9 #Dataset je k dispozici na https://www.kaggle.com/datasets/ruslankl/early-biomarkers-of-parkinsons-disease
10 rawDF = dataset
11
12 rawDF.drop(rawDF.columns[12:41], axis=1, inplace=True)
13
14 names = ["Typ", "A", "G", "PHP", "ADO", "DDFS", "AT", "APM", "AP", "BM", "LDE",
15          "CL", "EST", "RST", "AST", "DPI", "DVI", "GVI", "DUS", "DUF", "RLR", "PIR",
16          "RSR", "LRE", "EST2", "RST2", "AST2", "DPI2", "DVI2", "GVI2", "DUS2",
17          "DUF2", "RLR2", "PIR2", "RSR2", "LRE2"]
18 rawDF.columns = names
19
20 import pandas as pd
21
22 rawDF['Typ'] = rawDF['Typ'].str[0]
23
24 DF = rawDF.loc[:, rawDF.nunique() != 1]
25
26 DF['AT'] = DF['AT'].replace({"No": 0, "Yes": 1})
27 DF['BM'] = DF['BM'].replace({"No": 0, "Yes": 1})
28
29 df = DF[DF['Typ'] != "R"]
30
31 df['Parkinson'] = (df['Typ'] == "P").astype(int)
32
33 cols = list(df.columns)
34 cols.insert(1, cols.pop(cols.index('Parkinson')))
35 df = df[cols]
36
37 print(df['Parkinson'].value_counts())
38
39 df.drop(df.columns[[4, 5, 6]], axis=1, inplace=True)
40 df.drop(columns=['Typ'], inplace=True)
41
42 df['AT'] = pd.Categorical(df['AT'], categories=[0, 1], ordered=True)
43 df['BM'] = pd.Categorical(df['BM'], categories=[0, 1], ordered=True)
```

```
43 df['CL'] = pd.Categorical((df['CL'] == 0).astype(int))
44
45 df.drop(df.columns[2:6], axis=1, inplace=True)
46
47 df #Zobrazeni vysledneho datasetu
```


B. Vizualizace dat na datasetu Early Biomarkers of Parkinson's Disease

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 #Vytvoreni histogramu rozdeleni hodnot cilove promenne
4 plt.figure(figsize=(8, 6))
5 sns.countplot(x='Parkinson', data=df)
6 plt.xlabel('Parkinson (0 = Zdrav , 1 = M Parkinsonovou nemoc)')
7 plt.ylabel('Počet')
8 plt.show()
9
10 cor = df.corr()
11
12 # Vizualizace korelacnich koeficientu s vyuzitim heatmap
13 plt.figure(figsize=(14, 8))
14 sns.heatmap(cor, annot=True, cmap='coolwarm', fmt=".2f", cbar_kws={'label': 'Correlation Coefficient'})
15 plt.show()
```

C. Rozdělení na trenovací a testovací sady a použití modelů strojového učení

```
1 import numpy as np
2 import pandas as pd
3
4 #sklearn
5 from sklearn.model_selection import train_test_split
6 from sklearn.feature_selection import mutual_info_classif
7 from sklearn.preprocessing import MinMaxScaler
8 #qiskit
9 from qiskit import *
10 from qiskit_algorithms.utils import algorithm_globals
11
12
13 X = df.drop(['Parkinson'],axis=1)
14 X_normalized = MinMaxScaler((0,1)).fit_transform(X)
15 X_normalized_df = pd.DataFrame(X_normalized, columns=X.columns)
16 columns_to_select = [ 'A', 'RST', 'AST', 'DPI', 'DVI', 'DUS', 'RST2', 'DPI2',
17                       'DVI2', 'DUS2', 'DUF2', 'PIR2']
18 selected_features = X_normalized_df[columns_to_select]
19 y = df['Parkinson']
20
21 random_state = algorithm_globals.random_seed = 123
22 X_train, X_test, y_train, y_test = train_test_split(selected_features, y,
23 test_size=0.20, random_state=random_state)
24
25 from qiskit_ibm_runtime import QiskitRuntimeService,Session
26 from qiskit.primitives import BackendSampler
27 from qiskit_aer import Aer
28
29 from sqlearn.encoding_circuit import (
30 YZ_CX_EncodingCircuit)
31 from sqlearn.kernel import FidelityKernel, QSVK
32 from sqlearn.util import Executor
33 from qiskit_ibm_runtime import Sampler as RuntimeSampler
34
35 executor = Executor(BackendSampler(Aer.get_backend("statevector_simulator")))
36 #Inicializace executoru, ktery pouziva BackendSampler
37
38 #s simulatorem stavoveho vektoru pro vypocty
39
40 #Vytvari kvantovy obvod pro kodovani vstupnich dat do kvantoveho stavu.
41 #Obvod pouziva 12 qubitu, 12 vstupnich rysu a je rozdelen do 2 vrstev.
42 feature_map = YZ_CX_EncodingCircuit(
43 num_qubits=12, num_features=12, num_layers=2)
44
45 # Inicializuje kvantovy kernel pomoci vyse definovaneho kvantoveho obvodu a
```

```

    executoru.
45 q_kernel = FidelityKernel(
46 feature_map, executor=executor)
47
48 #Vytvori kvantovy support vector machine klasifikator s pouzitim kvantoveho
    kernelu
49 model_qsvc = QSVC(q_kernel, probability=True)
50
51 #Trenovani modelu
52 model_qsvc.fit(X_train, y_train)
53
54 # Vytvoreni predikce
55 y_pred_sqlearn = model_qsvc.predict(X_test)
56 y_pred_sqlearn
57
58 #Vystup: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0])
59
60
61 from sklearn.metrics import precision_score, recall_score, f1_score
62
63 # Vypocet presnosti, uplnosti a F1 skore
64 precision = precision_score(y_test, y_pred_sqlearn, average='micro')
65 recall = recall_score(y_test, y_pred_sqlearn, average='micro')
66 f1 = f1_score(y_test, y_pred_sqlearn, average='micro')
67
68
69
70 # Vystup vypoctenych metrik
71 print(f"Precision: {precision:.3f}")
72 print(f"Recall: {recall:.3f}")
73 print(f"F1 Score: {f1:.3f}")
74
75 #Vystup metrik:
76 #Precision: 0.812
77 #Recall: 0.812
78 #F1 Score: 0.812
79
80
81 from sklearn.metrics import precision_score, recall_score, f1_score
82
83 # Vypocet presnosti, uplnosti a F1 skore
84 precision = precision_score(y_test, y_pred_sqlearn, average='macro')
85 recall = recall_score(y_test, y_pred_sqlearn, average='macro')
86 f1 = f1_score(y_test, y_pred_sqlearn, average='macro')
87
88
89
90 # Vystup vypoctenych metrik
91 print(f"Precision: {precision:.3f}")
92 print(f"Recall: {recall:.3f}")
93 print(f"F1 Score: {f1:.3f}")
94
95 #Precision: 0.900

```

```

96 #Recall: 0.625
97 #F1 Score: 0.644
98
99 from sklearn.metrics import accuracy_score
100 accuracy = accuracy_score(y_test, y_pred_sqlearn)
101 #Vystup spravnosti modelu
102 print(f"Accuracy Score: {accuracy:.3f}")
103
104 #Accuracy Score: 0.812
105 from sklearn.metrics import confusion_matrix
106 import matplotlib.pyplot as plt
107 import seaborn as sns
108
109 # Matice zamen pro QSVC model z knihovny sqlearn
110 cm = confusion_matrix(y_test, y_pred_sqlearn)
111 sns.heatmap(cm, annot=True, fmt='d')
112 plt.title('Confusion Matrix')
113 plt.ylabel('Actual')
114 plt.xlabel('Predicted')
115 plt.show()
116
117 #Pouziti SVC algoritmu
118
119 import numpy as np
120 from sklearn.pipeline import make_pipeline
121 from sklearn.preprocessing import StandardScaler
122 from sklearn.svm import SVC
123
124 # Vytvo en klasifika n ho modelu
125 clf = SVC(probability=True)
126
127 # Tr nov n modelu
128 clf.fit(X_train, y_train)
129
130 #Predikce modelu
131 y_pred_svc = clf.predict(X_test)
132 y_pred_svc
133
134 #array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0])
135
136
137 from sklearn.metrics import precision_score, recall_score, f1_score
138
139 #Vypocet presnosti, uplnosti a F1 skore
140 precision = precision_score(y_test, y_pred_svc, average='micro')
141 recall = recall_score(y_test, y_pred_svc, average='micro')
142 f1 = f1_score(y_test, y_pred_svc, average='micro')
143
144 #Vystup vypoctenych metrik
145 print(f"Precision: {precision:.3f}")
146 print(f"Recall: {recall:.3f}")
147 print(f"F1 Score: {f1:.3f}")
148

```

```

149 #Precision: 0.750
150 #Recall: 0.750
151 #F1 Score: 0.750
152
153
154 # Vypocet presnosti, uplnosti a F1 skore
155 precision = precision_score(y_test, y_pred_svc, average='macro')
156 recall = recall_score(y_test, y_pred_svc, average='macro')
157 f1 = f1_score(y_test, y_pred_svc, average='macro')
158
159 # Vystup vypoctenych metrik
160 print(f"Precision: {precision:.3f}")
161 print(f"Recall: {recall:.3f}")
162 print(f"F1 Score: {f1:.3f}")
163
164 #Precision: 0.643
165 #Recall: 0.583
166 #F1 Score: 0.590
167
168 from sklearn.metrics import accuracy_score
169 accuracy = accuracy_score(y_test, y_pred_svc)
170
171 print(f"Accuracy Score: {accuracy:.3f}")
172
173 #Vystup spravnosti modelu:Accuracy Score: 0.750
174
175 # Matice zamen pro SVC model z knihovny scikit-learn
176 cm = confusion_matrix(y_test, y_pred_svc)
177 sns.heatmap(cm, annot=True, fmt='d')
178 plt.title('Confusion Matrix')
179 plt.ylabel('Actual')
180 plt.xlabel('Predicted')
181 plt.show()
182
183 #Pouziti QSVC algoritmu z knihovny qiskit
184 from qiskit.primitives import Sampler
185 from qiskit.circuit.library import ZZFeatureMap
186 from qiskit import QuantumCircuit
187 from qiskit_algorithms.state_fidelities import ComputeUncompute
188 from qiskit_machine_learning.kernels import FidelityQuantumKernel,
    FidelityStatevectorKernel
189 from qiskit.quantum_info import Statevector
190 from qiskit_machine_learning.algorithms import QSVC
191
192
193 feature_map = ZZFeatureMap(feature_dimension=12, reps=3)
194
195
196 sampler = Sampler()
197
198
199 fidelity = ComputeUncompute(sampler=sampler)
200

```

```

201 parkinsons_kernel = FidelityStatevectorKernel(feature_map=feature_map,
    statevector_type = Statevector, shots = 800, auto_clear_cache=True,)
202
203 qsvc_2 = QSVC(quantum_kernel=parkinsons_kernel, probability=True)
204 qsvc_2.fit(X_train, y_train)
205
206 y_pred_qiskit = qsvc_2.predict(X_test)
207 y_pred_qiskit
208
209 #array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
210
211 # Matice zamen pro QSVC model z knihovny qiskit
212 cm = confusion_matrix(y_test, y_pred_qiskit)
213 sns.heatmap(cm, annot=True, fmt='d')
214 plt.title('Confusion Matrix')
215 plt.ylabel('Actual')
216 plt.xlabel('Predicted')
217 plt.show()
218
219 #Predzpracovani dat pro QSVC model zvoleny na pocet atributu 5
220
221 from sklearn.feature_selection import mutual_info_classif
222 import numpy as np
223 from sklearn.preprocessing import MinMaxScaler
224 X = df.drop(['Parkinson'], axis=1)
225 X_normalized = MinMaxScaler((0,1)).fit_transform(X)
226 X_normalized_df = pd.DataFrame(X_normalized, columns=X.columns)
227 columns_to_select = ['DPI', 'DVI', 'RST2', 'DPI2', 'DVI2']
228 selected_features = X_normalized_df[columns_to_select]
229 y = df['Parkinson']
230 from qiskit_algorithms.utils import algorithm_globals
231 random_state = algorithm_globals.random_seed = 123
232 X_train, X_test, y_train, y_test = train_test_split(selected_features, y,
233 test_size=0.20, random_state=random_state)
234
235 #QSVC model
236
237 feature_map = ZZFeatureMap(feature_dimension=5, reps=3)
238
239
240 sampler = Sampler()
241
242
243 fidelity = ComputeUncompute(sampler=sampler)
244
245 parkinsons_kernel = FidelityStatevectorKernel(feature_map=feature_map,
    statevector_type = Statevector, shots = 800, auto_clear_cache=True,)
246
247 qsvc_2 = QSVC(quantum_kernel=parkinsons_kernel, probability=True)
248 qsvc_2.fit(X_train, y_train)
249
250
251 y_pred_qiskit = qsvc_2.predict(X_test)

```

```

252 y_pred_qiskit
253
254 #array([0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0])
255
256 # Vypocet presnosti, uplnosti a F1 skore
257 precision = precision_score(y_test, y_pred_qiskit, average='micro')
258 recall = recall_score(y_test, y_pred_qiskit, average='micro')
259 f1 = f1_score(y_test, y_pred_qiskit, average='micro')
260
261 # Vystup metrik
262 print(f"Precision: {precision:.3f}")
263 print(f"Recall: {recall:.3f}")
264 print(f"F1 Score: {f1:.3f}")
265
266 #Precision: 0.875
267 #Recall: 0.875
268 #F1 Score: 0.875
269
270 # Calculate precision, recall, and F1 score
271 precision = precision_score(y_test, y_pred_qiskit, average='macro')
272 recall = recall_score(y_test, y_pred_qiskit, average='macro')
273 f1 = f1_score(y_test, y_pred_qiskit, average='macro')
274
275 # Print the calculated metrics with three decimal places
276 print(f"Precision: {precision:.3f}")
277 print(f"Recall: {recall:.3f}")
278 print(f"F1 Score: {f1:.3f}")
279
280 #Precision: 0.833
281 #Recall: 0.833
282 #F1 Score: 0.833
283
284 from sklearn.metrics import accuracy_score
285 accuracy = accuracy_score(y_test, y_pred_qiskit)
286 #Vypocet spravnosti modelu QSVC
287 print(f"Accuracy Score: {accuracy:.3f}")
288 #Accuracy Score: 0.875
289
290 # Matice zamen pro QSVC model z knihovny qiskit
291 cm = confusion_matrix(y_test, y_pred_qiskit)
292 sns.heatmap(cm, annot=True, fmt='d')
293 plt.title('Confusion Matrix')
294 plt.ylabel('Actual')
295 plt.xlabel('Predicted')
296 plt.show()

```

D. Použití SHAP metody

```
1 #inicializace shap knihovny
2 import shap
3
4 #prevod testovacich a trenovacich dat na dataframe
5 X_train_df = pd.DataFrame(X_train)
6 X_test_df = pd.DataFrame(X_test)
7
8 #Inicializace KernelExplainer pro SVC model
9 explainer = shap.KernelExplainer(clf.predict_proba, X_train_df)
10
11 shap_values = explainer.shap_values(X_test_df.iloc[0, :])
12
13 features= [ 'RST', 'AST', 'DPI', 'DVI', 'DUS', 'DVI2', 'DUS2','DUF2','PIR2']
14
15 #Inicializace shap force plot pro SVC model
16 shap.force_plot(explainer.expected_value[0], shap_values[:, 0], X_test_df.
17                 iloc[0, :], feature_names=features,matplotlib=True)
18
19 #SPusteni KernelExplainer pro QSVC z knihovny sklearn
20 explainer = shap.KernelExplainer(model_qsvc.predict_proba, X_train_df)
21
22 shap_values = explainer.shap_values(X_test_df.iloc[0, :])
23
24 features= [ 'RST', 'AST', 'DPI', 'DVI', 'DUS', 'DVI2', 'DUS2','DUF2','PIR2']
25 shap.force_plot(explainer.expected_value[0], shap_values[:, 0], X_test_df.
26                 iloc[0, :], feature_names=features,matplotlib=True)
27
28 #Spusteni KernelExplainer pro QSVC z knihovny qiskit
29 explainer = shap.KernelExplainer(qsvc_2.predict_proba, X_train_df)
30
31 shap_values = explainer.shap_values(X_test_df.iloc[0, :])
32
33 features= ['DPI', 'DVI', 'RST2', 'DPI2', 'DVI2']
34 #Zobrazeni SHAP force plot
35 shap.force_plot(explainer.expected_value[0], shap_values[:, 0], X_test_df.
36                 iloc[0, :], feature_names=features,show=False,matplotlib=True)
```


E. Příprava dat pro Dataset Oxford Parkinson's Disease Detection Dataset

```
1 import pandas as pd
2 from sklearn.preprocessing import MinMaxScaler
3 url = "http://archive.ics.uci.edu/ml/machine-learning-databases/parkinsons/
   parkinsons.data"
4
5 #Nacteni dat
6 parkinsons_data = pd.read_csv(url)
7 X = parkinsons_data.drop(['name', 'status'], axis=1)
8 y = parkinsons_data['status']
9 parkinsons_data
10
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13
14 #Vytvoreni histogramu rozdeni hodnot cilove promenne
15 parkinsons_data = pd.read_csv(url)
16 plt.figure(figsize=(8, 6))
17 sns.countplot(x='status', data=parkinsons_data)
18 plt.xlabel('Status (0 = Zdrav , 1 = M Parkinsonovou nemoc)')
19 plt.ylabel('Po et')
20 plt.show()
21
22
23 #Vyuziti metody SMOTE
24 from imblearn.over_sampling import SMOTE
25 sm = SMOTE(random_state=42)
26 X, y = sm.fit_resample(X, y)
27
28
29 from sklearn.feature_selection import mutual_info_classif
30 import numpy as np
31
32 mi = mutual_info_classif(X, y)
33 mi /= np.max(mi) # Normalizace hodnot vzajemneho vyberu priznaku
34
35 # Volba atributu s hodnotou vetsi nez 0.79
36 selected_features = X.columns[mi > 0.79]
37
38 selected_features
39
40 X_sel = X[selected_features]
41
42 #Vytvoreni grafu vzajemne informace pro vyber priznaku
43
44 import pandas as pd
45 import numpy as np
46 from sklearn.feature_selection import mutual_info_classif
```

```

47 import altair as alt
48
49
50 #Vypocet mutual information
51 mi = mutual_info_classif(X, y)
52 mi_normalized = mi / np.max(mi)
53
54 # Prevod do DataFrame
55 mi_df = pd.DataFrame({'Feature': X.columns, 'Mutual Information':
    mi_normalized})
56
57 # Nastaveni threshold
58 threshold = 0.79
59
60 # Vytvoreni bar plot
61 bar_chart = alt.Chart(mi_df).mark_bar().encode(
62     x='Mutual Information',
63     y='Feature',
64     tooltip=['Feature', 'Mutual Information']
65 ).properties(
66     width=600,
67     height=400,
68     title='Vz jemn informace pro vlastnosti dat Parkinsonovy nemoci'
69 )
70
71 # Vytvoreni horizontalni cary pro nastaveni threshold
72 threshold_line = alt.Chart(pd.DataFrame({'Mutual Information': [threshold]}))
    .mark_rule(color='red').encode(
73     x='Mutual Information:Q'
74 )
75
76 # Kombinace bar chart spolu s threshold line
77 alt_chart = (bar_chart + threshold_line).configure_axis(
78     labelFontSize=12,
79     titleFontSize=12
80 )
81
82 # Vystup grafu
83 alt_chart
84
85 #Vytvoreni korelacni analyzy
86 X['status'] = y
87
88 correlation_matrix = X.corr()
89
90 # Vytvoreni heatmapy pro zobrazeni korelaci
91 plt.figure(figsize=(12, 10))
92 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
    linewidths=.5)
93 plt.title('Heatmapa korelaci mezi vlastnostmi Parkinsonovy nemoci')
94 plt.show()
95
96 X.drop('status', axis=1, inplace=True)

```

F. Rozdělení dat na trenovací a testovací sady a použití jednotlivých modelů

```
1 #Rozdeleni dat, kde 80% dat se pouzije pro trenovaci sadu
2 from qiskit_algorithms.utils import algorithm_globals
3 from sklearn.model_selection import train_test_split
4 algorithm_globals.random_seed = 123
5 X_train, X_valid, y_train, y_valid = train_test_split(
6     X_sel, y, train_size=0.8, random_state=algorithm_globals.random_seed
7 )
8 input_shape = X_train.shape[1]
9
10 from qiskit.circuit.library import ZZFeatureMap
11
12 feature_map = ZZFeatureMap(feature_dimension=input_shape, reps=1)
13 #feature_map.decompose().draw(output="mpl", fold=20)
14 #s pomoci decompose zobrazime obvod ZZFeatureMap
15
16
17 from qiskit import QuantumCircuit
18 from qiskit.primitives import Sampler
19 from qiskit_algorithms.state_fidelities import ComputeUncompute
20 from qiskit.circuit.library import ZZFeatureMap
21 from qiskit_machine_learning.kernels import FidelityQuantumKernel
22 from qiskit_aer import Aer
23 from sklearn.pipeline import make_pipeline
24
25 sampler = Sampler()
26
27 fidelity = ComputeUncompute(sampler=sampler)
28
29 parkinsons_kernel = FidelityQuantumKernel(fidelity=fidelity, feature_map=
    feature_map)
30
31
32 from qiskit_machine_learning.algorithms import QSVC
33 qsvc = make_pipeline(MinMaxScaler(), QSVC(quantum_kernel=parkinsons_kernel,
    probability=True))
34 qsvc.fit(X_train, y_train)
35 qsvc.predict(X_valid)
36 #array([0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
37 #       1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0,
38 #       1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0], dtype=int64)
39
40 qsvc.score(X_valid, y_valid)
41 # 0.9491525423728814
42
```

```

43 #Zobrazeni klasicke matice zamen
44 from sklearn.metrics import confusion_matrix, classification_report,
    roc_curve, auc
45 import matplotlib.pyplot as plt
46 import seaborn as sns
47
48 # Vytvoreni predikce
49 y_pred = qsvc.predict(X_valid)
50
51 # Spravnost modelu
52 accuracy = qsvc.score(X_valid, y_valid)
53 print("Model Accuracy:", accuracy)
54
55 # Zobrazeni matice zamen
56 cm = confusion_matrix(y_valid, y_pred)
57 sns.heatmap(cm, annot=True, fmt='d')
58 plt.title('Confusion Matrix')
59 plt.ylabel('Actual')
60 plt.xlabel('Predicted')
61 plt.show()
62
63 # Klasifikacni Report
64 print(classification_report(y_valid, y_pred))
65
66 # ROC Krivka a AUC (pokud binarni klasifikace)
67 if len(np.unique(y)) == 2:
68
69     y_pred_proba = qsvc.predict_proba(X_valid)[:,1]
70     fpr, tpr, _ = roc_curve(y_valid, y_pred_proba)
71     roc_auc = auc(fpr, tpr)
72
73     plt.figure()
74     plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area =
    %0.2f)' % roc_auc)
75     plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
76     plt.xlim([0.0, 1.0])
77     plt.ylim([0.0, 1.05])
78     plt.xlabel('False Positive Rate')
79     plt.ylabel('True Positive Rate')
80     plt.title('Receiver Operating Characteristic')
81     plt.legend(loc="lower right")
82     plt.show()
83     plt.savefig("qsvc_new_graph.png")
84
85 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
    predicting-a-stroke-shap-lime-explainer-eli5
86 from sklearn.metrics import confusion_matrix, f1_score, accuracy_score,
    recall_score, precision_score, roc_auc_score
87 import seaborn as sns
88 import matplotlib.pyplot as plt
89 import matplotlib.colors
90 import pandas as pd
91

```

```

92
93 qsvc_cm = confusion_matrix(y_valid, y_pred)
94 qsvc_f1 = f1_score(y_valid, y_pred)
95 qsvc_accuracy = accuracy_score(y_valid, y_pred)
96 qsvc_recall = recall_score(y_valid, y_pred)
97 qsvc_precision = precision_score(y_valid, y_pred)
98 qsvc_roc_auc = roc_auc_score(y_valid, y_pred_proba)
99 y_pred_proba for ROC AUC Score
100
101
102 qsvc_df = pd.DataFrame({
103     'Q SVC Score': [qsvc_f1, qsvc_accuracy, qsvc_recall, qsvc_precision,
104                     qsvc_roc_auc],
105     'Metrics': ["F1", "Accuracy", "Recall", "Precision", "ROC AUC Score"]
106 }).set_index('Metrics')
107
108 colors = ["#e3f2fd", "#64b5f6", "#0d47a1"] # byly pouzity barvy Light blue,
109         medium blue, deep blue
110 colormap = matplotlib.colors.LinearSegmentedColormap.from_list("CustomBlue",
111         colors)
112 background_color = "#eceff1"
113
114 # Vytvoreni grafu
115 fig = plt.figure(figsize=(10, 8))
116 gs = fig.add_gridspec(4, 2)
117 gs.update(wspace=0.1, hspace=0.5)
118 ax0 = fig.add_subplot(gs[0, :])
119 ax1 = fig.add_subplot(gs[1, :])
120
121 sns.heatmap(qsvc_df.T, cmap=colormap, annot=True, fmt=".1%", vmin=0, vmax
122             =0.95, yticklabels='', linewidths=2.5, cbar=False, ax=ax0, annot_kws={"
123             fontsize": 12})
124 fig.patch.set_facecolor(background_color)
125 ax0.set_facecolor(background_color)
126
127 # Heatmap pro matici zamen
128 sns.heatmap(qsvc_cm, cmap=colormap, annot=True, fmt="d", linewidths=5, cbar=
129             False, ax=ax1,
130             yticklabels=['Actual Negative', 'Actual Positive'], xticklabels=['
131             Predicted Negative', 'Predicted Positive'], annot_kws={"fontsize": 12})
132 ax1.tick_params(axis='both', which='both', length=0)
133 ax1.set_facecolor(background_color)
134
135 plt.show()
136 plt.savefig("qsvc_new_graph_bp.png")
137
138
139 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
140 predicting-a-stroke-shap-lime-explainer-eli5
141
142 import matplotlib.pyplot as plt

```

```

137 import seaborn as sns
138 import pandas as pd
139 from sklearn.inspection import permutation_importance
140 from sklearn.preprocessing import MinMaxScaler
141 from sklearn.pipeline import make_pipeline
142 import numpy as np
143
144
145 qsvc = make_pipeline(MinMaxScaler(), QSVK(quantum_kernel=parkinsons_kernel,
146     probability=True))
147 qsvc.fit(X_train, y_train)
148
149 # Vypocet permutation performance
150 result = permutation_importance(qsvc, X_valid, y_valid, n_repeats=10,
151     random_state=42, n_jobs=-1)
152 sorted_idx = result.importances_mean.argsort()
153
154 # Priprava dat pro vizualizaci
155 fi = pd.DataFrame({
156     'Feature': X_train.columns[sorted_idx],
157     'Importance': result.importances_mean[sorted_idx]
158 })
159
160 background_color = "#fbfbfb"
161 fig, ax = plt.subplots(1, 1, figsize=(10, 8), facecolor=background_color)
162
163 # Vytvoreni color map a bar plotu
164 color_map = ['lightgray' for _ in range(len(fi))]
165 color_map[-3:] = ['#0f4c81', '#0f4c81', '#0f4c81']
166 sns.barplot(data=fi, x='Importance', y='Feature', ax=ax, palette=color_map)
167
168 ax.set_facecolor(background_color)
169 for s in ['top', 'left', 'right']:
170     ax.spines[s].set_visible(False)
171
172 fig.text(0.12, 0.92, "Feature Importance: QSVK Model for Parkinson's
173     Prediction", fontsize=18, fontweight='bold', fontfamily='serif')
174 plt.xlabel(" ", fontsize=12, fontweight='light', fontfamily='serif')
175 plt.ylabel(" ", fontsize=12, fontweight='light', fontfamily='serif')
176
177 ax.tick_params(axis='both', which='both', length=0)
178
179 import matplotlib.lines as lines
180 l1 = lines.Line2D([0.98, 0.98], [0, 1], transform=fig.transFigure, figure=fig,
181     color='black', lw=0.2)
182 fig.lines.extend([l1])
183 plt.show()

```

G. Použití QSVC algoritmu z knihovny sqlearn

```
1 from qiskit_ibm_runtime import QiskitRuntimeService, Session
2 from qiskit.primitives import BackendSampler
3 from qiskit_aer import Aer
4
5 from sqlearn.encoding_circuit import (
6     YZ_CX_EncodingCircuit)
7 from sqlearn.kernel import FidelityKernel, QSVC
8 from sqlearn.util import Executor
9 from qiskit_ibm_runtime import Sampler as RuntimeSampler
10
11 executor = Executor(BackendSampler(Aer.get_backend("statevector_simulator")))
12
13 feature_map = YZ_CX_EncodingCircuit(
14     num_qubits=7, num_features=7, num_layers=2)
15 q_kernel = FidelityKernel(
16     feature_map, executor=executor)
17
18 model_qsvc = QSVC(q_kernel, probability=True)
19
20
21
22 model_qsvc.fit(X_train, y_train)
23
24 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
25     predicting-a-stroke-shap-lime-explainer-eli5
26
27 import matplotlib.pyplot as plt
28 import seaborn as sns
29 import pandas as pd
30 from sklearn.inspection import permutation_importance
31 from sklearn.preprocessing import MinMaxScaler
32 from sklearn.pipeline import make_pipeline
33 import numpy as np
34 from qiskit_ibm_runtime import QiskitRuntimeService, Session
35 from qiskit.primitives import BackendSampler
36 from qiskit_aer import Aer
37 from sqlearn.encoding_circuit import (
38     YZ_CX_EncodingCircuit)
39 from sqlearn.kernel import FidelityKernel, QSVC
40 from sqlearn.util import Executor
41 from qiskit_ibm_runtime import Sampler as RuntimeSampler
42
43
44 model_qsvc = QSVC(q_kernel, probability=True)
45 model_qsvc.fit(X_train, y_train)
46
```

```

47 # Vypocet permutation performace
48 result = permutation_importance(model_qsvc, X_valid, y_valid, n_repeats=10,
    random_state=42, n_jobs=1)
49 sorted_idx = result.importances_mean.argsort()
50
51 # Priprava dat pro vizualizaci
52 fi = pd.DataFrame({
53     'Feature': X_train.columns[sorted_idx],
54     'Importance': result.importances_mean[sorted_idx]
55 })
56
57 background_color = "#fbfbfb"
58 fig, ax = plt.subplots(1, 1, figsize=(10, 8), facecolor=background_color)
59
60 # Vtvoreni color map a bar plotu
61 color_map = ['lightgray' for _ in range(len(fi))]
62 color_map[-3:] = ['#0f4c81', '#0f4c81', '#0f4c81']
63 sns.barplot(data=fi, x='Importance', y='Feature', ax=ax, palette=color_map)
64
65 ax.set_facecolor(background_color)
66 for s in ['top', 'left', 'right']:
67     ax.spines[s].set_visible(False)
68
69 fig.text(0.12, 0.92, "Feature Importance: QSVC Model for Parkinson's
    Prediction using squllearn", fontsize=18, fontweight='bold', fontfamily='
    serif')
70 plt.xlabel(" ", fontsize=12, fontweight='light', fontfamily='serif')
71 plt.ylabel(" ", fontsize=12, fontweight='light', fontfamily='serif')
72
73
74
75 ax.tick_params(axis='both', which='both', length=0)
76
77 # Vutvoreni cary pro nasledne rozde;eni
78 import matplotlib.lines as lines
79 l1 = lines.Line2D([0.98, 0.98], [0, 1], transform=fig.transFigure, figure=fig
    , color='black', lw=0.2)
80 fig.lines.extend([l1])
81
82 plt.show()
83
84
85 from sklearn.metrics import confusion_matrix, classification_report,
    roc_curve, auc
86 import matplotlib.pyplot as plt
87 import seaborn as sns
88 # Znazorneni klasicke matice zamen, ROC AUC krivky a classification report
89 # Vytvoreni predikce
90 y_pred = model_qsvc.predict(X_valid)
91
92 accuracy = model_qsvc.score(X_valid, y_valid)
93 print("Model Accuracy:", accuracy)
94

```



```

95 #Zobrazeni matici zamen
96 cm = confusion_matrix(y_valid, y_pred)
97 sns.heatmap(cm, annot=True, fmt='d')
98 plt.title('Confusion Matrix')
99 plt.ylabel('Actual')
100 plt.xlabel('Predicted')
101 plt.show()
102
103 # Vytvoreni classification report
104 print(classification_report(y_valid, y_pred))
105
106 # Vytvoreni ROC krivky a AUC (pokud binarni klasifikace)
107 if len(np.unique(y)) == 2:
108
109     y_pred_proba = model_qsvc.predict_proba(X_valid)[::,1]
110     fpr, tpr, _ = roc_curve(y_valid, y_pred_proba)
111     roc_auc = auc(fpr, tpr)
112
113     plt.figure()
114     plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area =
115 %0.2f)' % roc_auc)
116     plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
117     plt.xlim([0.0, 1.0])
118     plt.ylim([0.0, 1.05])
119     plt.xlabel('False Positive Rate')
120     plt.ylabel('True Positive Rate')
121     plt.title('Receiver Operating Characteristic')
122     plt.legend(loc="lower right")
123     plt.show()
124     plt.savefig("qsvc_new_graph.png")
125
126 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
127 predicting-a-stroke-shap-lime-explainer-eli5
128
129 from sklearn.metrics import confusion_matrix, f1_score, accuracy_score,
130 recall_score, precision_score, roc_auc_score
131 import seaborn as sns
132 import matplotlib.pyplot as plt
133 import matplotlib.colors
134 import pandas as pd
135
136 model_qsvc_cm = confusion_matrix(y_valid, y_pred)
137 model_qsvc_f1 = f1_score(y_valid, y_pred)
138 model_qsvc_accuracy = accuracy_score(y_valid, y_pred)
139 model_qsvc_recall = recall_score(y_valid, y_pred)
140 model_qsvc_precision = precision_score(y_valid, y_pred)
141 model_qsvc_roc_auc = roc_auc_score(y_valid, y_pred_proba)
142
143 # Prevod na dataframe
144 model_qsvc_df = pd.DataFrame({
145     'Q SVC Score': [model_qsvc_f1, model_qsvc_accuracy, model_qsvc_recall,
146 model_qsvc_precision, model_qsvc_roc_auc],

```

```

144     'Metrics': ["F1", "Accuracy", "Recall", "Precision", "ROC AUC Score"]
145 }).set_index('Metrics')
146
147 # Nastaveni barev
148 colors = ["#e3f2fd", "#64b5f6", "#0d47a1"]
149 colormap = matplotlib.colors.LinearSegmentedColormap.from_list("CustomBlue",
150     colors)
151 background_color = "#eceff1"
152
153 # Vytvoreni grafu
154 fig = plt.figure(figsize=(10, 8))
155 gs = fig.add_gridspec(4, 2)
156 gs.update(wspace=0.1, hspace=0.5)
157 ax0 = fig.add_subplot(gs[0, :])
158 ax1 = fig.add_subplot(gs[1, :])
159
160 sns.heatmap(model_qsvc_df.T, cmap=colormap, annot=True, fmt=".1%", vmin=0,
161     vmax=0.95, yticklabels='', linewidths=2.5, cbar=False, ax=ax0, annot_kws={
162     "fontsize": 12})
163 fig.patch.set_facecolor(background_color)
164 ax0.set_facecolor(background_color)
165
166 # Heatmap pro matici zamen
167 sns.heatmap(model_qsvc_cm, cmap=colormap, annot=True, fmt="d", linewidths=5,
168     cbar=False, ax=ax1,
169     yticklabels=['Actual Negative', 'Actual Positive'], xticklabels=[
170     'Predicted Negative', 'Predicted Positive'], annot_kws={"fontsize": 12})
171 ax1.tick_params(axis='both', which='both', length=0)
172 ax1.set_facecolor(background_color)
173
174 plt.show()
175 plt.savefig("model_qsvc_new_graph_bp.png")

```

H. Použití SVC algoritmu z knihovny scikit-learn

```
1
2 from sklearn.svm import SVC
3 from sklearn.pipeline import make_pipeline
4
5 #Vytvoreni Pipeline s MinMaxScaler pro normalizaci hodnot a samotnym SVC
   algoritmem
6 svc = make_pipeline(MinMaxScaler(),SVC(probability=True))
7 #Trenovani modelu
8 svc.fit(X_train, y_train)
9
10 svc.predict(X_valid)
11 #array([0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1,
12 #       1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0,
13 #       1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0], dtype=int64)
14
15
16 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
   predicting-a-stroke-shap-lime-explainer-eli5
17
18 import matplotlib.pyplot as plt
19 import seaborn as sns
20 import pandas as pd
21 from sklearn.inspection import permutation_importance
22 from sklearn.preprocessing import MinMaxScaler
23 from sklearn.svm import SVC
24 from sklearn.pipeline import make_pipeline
25 import numpy as np
26
27
28 # Vypocet permutation importance
29 result = permutation_importance(svc, X_valid, y_valid, n_repeats=10,
   random_state=42, n_jobs=-1)
30 sorted_idx = result.importances_mean.argsort()
31
32 # Priprava dat pro vizualizaci
33 fi = pd.DataFrame({
34     'Feature': X_train.columns[sorted_idx],
35     'Importance': result.importances_mean[sorted_idx]
36 })
37
38 background_color = "#fbfbfb"
39 fig, ax = plt.subplots(1, 1, figsize=(10, 8), facecolor=background_color)
40
41
42 color_map = ['lightgray' for _ in range(len(fi))]
43 color_map[-3:] = ['#0f4c81', '#0f4c81', '#0f4c81'] # Zvyrazneni top 3
   atributu
```

```

44 sns.barplot(data=fi, x='Importance', y='Feature', ax=ax, palette=color_map)
45
46 ax.set_facecolor(background_color)
47 for s in ['top', 'left', 'right']:
48     ax.spines[s].set_visible(False)
49
50 fig.text(0.12, 0.92, "Feature Importance: SVC Model for Parkinson's
    Prediction", fontsize=18, fontweight='bold', fontfamily='serif')
51 plt.xlabel(" ", fontsize=12, fontweight='light', fontfamily='serif')
52 plt.ylabel(" ", fontsize=12, fontweight='light', fontfamily='serif')
53
54
55
56 ax.tick_params(axis='both', which='both', length=0)
57
58
59 import matplotlib.lines as lines
60 l1 = lines.Line2D([0.98, 0.98], [0, 1], transform=fig.transFigure, figure=fig
    , color='black', lw=0.2)
61 fig.lines.extend([l1])
62
63 plt.show()
64
65
66 #Zde probiha inicializace klasicke matice zamen s ROC,AUC krivkami pro
    nasledne zdokonaleni s pomoci programovani UX grafu
67
68 from sklearn.metrics import confusion_matrix, classification_report,
    roc_curve, auc
69 import matplotlib.pyplot as plt
70 import seaborn as sns
71 import numpy as np
72 from sklearn.svm import SVC
73
74
75
76 #Znovu pouzijeme Trenovani modelu
77 svc.fit(X_train, y_train)
78 #Vytvoreni predikce
79 y_pred = svc.predict(X_valid)
80
81
82 # Matice Zamen
83 cm = confusion_matrix(y_valid, y_pred)
84 sns.heatmap(cm, annot=True, fmt='d')
85 plt.title('Confusion Matrix')
86 plt.ylabel('Actual')
87 plt.xlabel('Predicted')
88 plt.show()
89
90 # Vytvoreni Klasifikacniho reportu
91 print(classification_report(y_valid, y_pred))
92

```

```

93 # ROC krivka a AUC (pokud je to binarni klasifikace)
94 if len(np.unique(y)) == 2:
95
96     y_pred_proba = svc.predict_proba(X_valid)[::,1]
97     fpr, tpr, _ = roc_curve(y_valid, y_pred_proba)
98     roc_auc = auc(fpr, tpr)
99
100     plt.figure()
101     plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area =
102 %0.2f)' % roc_auc)
103     plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
104     plt.xlim([0.0, 1.0])
105     plt.ylim([0.0, 1.05])
106     plt.xlabel('False Positive Rate')
107     plt.ylabel('True Positive Rate')
108     plt.title('Receiver Operating Characteristic')
109     plt.legend(loc="lower right")
110     plt.show()
111
112
113 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
114     predicting-a-stroke-shap-lime-explainer-eli5
115
116 from sklearn.metrics import confusion_matrix, f1_score, accuracy_score,
117     recall_score, precision_score, roc_auc_score
118 import seaborn as sns
119 import matplotlib.pyplot as plt
120 import matplotlib.colors
121 import pandas as pd
122
123
124 svc_cm = confusion_matrix(y_valid, y_pred)
125 svc_f1 = f1_score(y_valid, y_pred)
126 svc_accuracy = accuracy_score(y_valid, y_pred)
127 svc_recall = recall_score(y_valid, y_pred)
128 svc_precision = precision_score(y_valid, y_pred)
129 svc_roc_auc = roc_auc_score(y_valid, y_pred_proba)
130
131 # Dataframe pro vizualizaci heatmap
132 svc_df = pd.DataFrame({
133     'QSVC Score': [svc_f1, svc_accuracy, svc_recall, svc_precision,
134     svc_roc_auc],
135     'Metrics': ["F1", "Accuracy", "Recall", "Precision", "ROC AUC Score"]
136 }).set_index('Metrics')
137
138 # Nastaveni barev
139 colors = ["#e3f2fd", "#64b5f6", "#0d47a1"] # Zde jsou znazornene barvy Light
140     blue, medium blue, deep blue
141 colormap = matplotlib.colors.LinearSegmentedColormap.from_list("CustomBlue",
142     colors)
143 background_color = "#eceff1" #Zde je nastavena barva na pozadi
144

```

```

140 # Vytvoreni grafu
141 fig = plt.figure(figsize=(10, 8))
142 gs = fig.add_gridspec(4, 2)
143 gs.update(wspace=0.1, hspace=0.5)
144 ax0 = fig.add_subplot(gs[0, :])
145 ax1 = fig.add_subplot(gs[1, :])
146
147
148 sns.heatmap(svc_df.T, cmap=colormap, annot=True, fmt=".1%", vmin=0, vmax
    =0.95, yticklabels='', linewidths=2.5, cbar=False, ax=ax0, annot_kws={"
    fontsize": 12})
149 fig.patch.set_facecolor(background_color)
150 ax0.set_facecolor(background_color)
151
152 # Heatmap pro matici zamen
153 sns.heatmap(svc_cm, cmap=colormap, annot=True, fmt="d", linewidths=5, cbar=
    False, ax=ax1,
154             yticklabels=['Actual Negative', 'Actual Positive'], xticklabels=[
    'Predicted Negative', 'Predicted Positive'], annot_kws={"fontsize": 12})
155 ax1.tick_params(axis='both', which='both', length=0)
156 ax1.set_facecolor(background_color)
157
158 plt.show()
159
160 #Tato cast byla inspirovana https://www.kaggle.com/code/joshuaswords/
    predicting-a-stroke-shap-lime-explainer-eli5
161
162 # Priprava dat
163 url = "http://archive.ics.uci.edu/ml/machine-learning-databases/parkinsons/
    parkinsons.data"
164 parkinsons_data = pd.read_csv(url)
165 X = parkinsons_data.drop(['name', 'status'], axis=1)
166 y = parkinsons_data['status']
167
168 # Zvolene atributy
169 variables_to_plot = ['MDVP:F0(Hz)', 'MDVP:F1(Hz)', 'MDVP:F2(Hz)', 'HNR']
170 labels = ['Pr m rn hlasov frekvence', 'Maxim ln hlasov frekvence',
    'Minim ln hlasov frekvence', 'Harmonicita']
171
172 # Nastaveni osy a grafu
173 fig, axs = plt.subplots(nrows=1, ncols=4, figsize=(22, 5), dpi=150, facecolor
    = '#fafafa')
174 fig.subplots_adjust(hspace=0.4, wspace=0.4)
175
176 # Iterace pres vsechny zvolene atributy
177 for i, var in enumerate(variables_to_plot):
178     ax = axs[i]
179     # Vytvoreni vizualizaci KDE pro Parkinson pozitivni a negativni pripady
180     sns.kdeplot(data=parkinsons_data[parkinsons_data['status'] == 1], x=var,
    ax=ax, color='#0f4c81', shade=True, label='Parkinsonova nemoc')
181     sns.kdeplot(data=parkinsons_data[parkinsons_data['status'] == 0], x=var,
    ax=ax, color='#9bb7d4', shade=True, label='Zdrav ')
182     ax.set_title(labels[i])

```

```
183     ax.set_facecolor('#fafafa')
184     ax.grid(True, linestyle=':', color='gray', dashes=(1, 5))
185     ax.legend()
186
187 # Pridani titlu
188 fig.suptitle('Rozdeleni hlasovych vlastnosti podle ciloveho atributu status',
189             fontsize=20, fontweight='bold', fontfamily='serif')
189 plt.show()
```